SENAI Limeira/SP
Curso Técnico em Desenvolvimento de Sistemas — 3º DEV-A



SENTINEL

Interface Conversacional Agêntica sobre Sistema MVC

Documentação técnica — versão 1.1

Arquitetura, agente, fluxos, segurança, código, dados, testes e rastreabilidade do sistema construído nas Fases 0 a 5

Equipe: Lucas Terminiello, Matheus Malaman, Jefferson Bruno C. Miguel, Júlio Cena

Limeira, outubro de 2026

Controle do documento

Campo	Valor
Documento	Sentinel — Documentação técnica
Versão	1.1 — 28 de outubro de 2026
Base do código	Repositório SENTINEL, branch `main`, revisão `ae79781`
Situação do projeto	Construção (Fases 0 a 5) concluída; teste manual e preparação da apresentação na Fase 6; congelamento de funcionalidades com data em aberto; entrega final em 25/11/2026
Substitui	Documentação 1.0 (9 de setembro de 2026), que registrava o conceito e a Fase 0
Fontes	`RELATORIO_FINAL_FASES_4_5.md`, código-fonte, suíte de testes e Notion (projeto SENTINEL)

Como ler este documento. As seções 1 a 3 apresentam o problema, o conceito e o público. As seções 4 a 7 descrevem o sistema construído: visão geral, arquitetura, o agente e os fluxos. As seções 8 a 10 tratam de segurança, do código e dos dados. As seções 11 a 13 cobrem interface, testes e a rastreabilidade de cada requisito. As seções 14 e 15 registram limitações e próximos passos.

Sumário

1. Contexto e motivação 5
1.1 Inviabilidade técnica do caminho original 5
1.2 Uma oportunidade mais forte 5
2. Conceito central 6
2.1 Exemplo de uso 6
2.2 Fundamentação teórica 6
2.3 Os quatro padrões de interação 6
3. Público-alvo e mercado 8
3.1 O tamanho e a natureza do mercado 8
3.2 O gargalo é a interface, não o acesso 8
3.3 Definição do público-alvo 8
3.4 Predições sobre o público-alvo 9
4. Visão geral do sistema construído 11
4.1 Stack tecnológica 11
4.2 Papéis e permissões 12
4.3 Telas 12
5. Arquitetura 13
5.1 As camadas 13
5.2 Componentes principais 14
5.3 Rotas 15
5.4 Decisões arquiteturais 16
5.5 Resiliência quando a IA falha 16
6. O agente 17
6.1 O catálogo de ferramentas 17
6.2 O que vai e o que nunca vai para a IA 17
6.3 Memória da conversa 18
6.4 Como a resposta do modelo é conferida 18
6.5 Tempo de resposta 19
7. Fluxos da aplicação 20
7.1 F1: do comando à gravação 20
7.2 Estados de uma proposta 22
7.3 Segurar para confirmar 23
7.4 F2: atualizar um registro 23
7.5 F3: lixeira, arquivamento e exclusão definitiva 24
7.6 F4: consultar e exportar 26
7.7 F5: modo manual, sem IA 27
8. Segurança 29
8.1 Princípios 29
8.2 Ameaças e proteções 30
8.3 Dados pessoais (LGPD) 31
8.4 Autenticação, sessão e limites 31
8.5 Ambiente e dependências 32
9. O código em detalhe 33
9.1 Uma tool no catálogo 33
9.2 A chamada ao provedor e a medição do tempo 34
9.3 Conferindo o que o modelo respondeu 35

9.4 Revisar: validar sem gravar 35
9.5 Confirmar: sem segredo antes da hora 36
9.6 Uso único do token 36
9.7 Execução e auditoria na mesma transação 37
9.8 RBAC com falha fechada 37
9.9 Regras exists presas ao tenant 38
9.10 Máscara LGPD 39
9.11 Higiene do histórico da conversa 39
9.12 Limite atômico de tentativas de senha 40
9.13 Bloqueio por dependência 41
9.14 CSV sem injeção de fórmula 41
9.15 Segurar para confirmar, no navegador 42
10. Modelo de dados e auditoria 43
10.1 A tabela de auditoria 44
10.2 Migrations 44
11. Interface e acessibilidade 46
11.1 Sinalização de risco 46
11.2 Acessibilidade 46
12. Qualidade e testes 47
12.1 Técnicas usadas 47
12.2 Teste manual 48
13. Requisitos e rastreabilidade 49
13.1 Requisitos funcionais 49
13.2 Requisitos não funcionais 50
13.3 RF03: ações em massa por plano de ações 50
14. Limitações reconhecidas 52
15. Próximos passos 53
15.1 Antes da entrega 53
15.2 Depois do TCC 53
Referências 54
Apêndice A. Glossário 55

1. Contexto e motivação

O Sentinel nasceu, no anteprojeto, como uma plataforma SaaS multi-tenant de detecção de anomalias e manutenção preditiva, apoiada em serviços de IA do Azure. No aprofundamento técnico da proposta, dois problemas concretos motivaram a revisão do conceito. Este documento registra o conceito revisado e, principalmente, **o sistema efetivamente construído**: arquitetura, agente, fluxos, segurança, código, dados, testes e rastreabilidade dos requisitos.

1.1 Inviabilidade técnica do caminho original

O Azure AI Anomaly Detector, serviço que embasava a camada preditiva do anteprojeto, está sendo descontinuado pela Microsoft. Desde setembro de 2023 não é possível criar novos recursos do serviço, e a aposentadoria definitiva ocorre em 1º de outubro de 2026, junto com o Metrics Advisor, também cogitado.

Além disso, manutenção preditiva de verdade exige histórico de falhas e treinamento de modelo, e um TCC de poucos meses não comporta, de forma realista, nem o tempo nem os dados que isso demanda.

1.2 Uma oportunidade mais forte

Da análise desses limites surgiu um conceito mais maduro e mais atual. Em vez de prever falhas em sensores, o sistema ataca um problema de usabilidade comum a todo software de gestão: a distância entre o que o usuário quer fazer e a navegação por menus, telas e cadastros necessária para fazê-lo. A camada preditiva foi substituída por uma **camada de interface conversacional agêntica**: o usuário declara a intenção em linguagem natural, e um agente de IA orquestra as operações de um sistema MVC tradicional, mantendo o humano no controle de toda decisão de escrita.

O domínio de aplicação escolhido é o de um **escritório de contabilidade** atendendo micro e pequenas empresas: lançamentos financeiros, categorias, clientes, fornecedores, funcionários e notas fiscais. O nome Sentinel foi mantido.

2. Conceito central

A ideia central cabe em uma frase: **um sistema de gestão em que o usuário declara a intenção em linguagem natural e um agente de IA orquestra as operações do sistema, mantendo o humano no controle das decisões.**

2.1 Exemplo de uso

Em vez de navegar por Cadastros → Lançamentos → Filtros, o usuário digita "quais lançamentos ainda estão pendentes?". O sistema responde com uma tabela que pode ser filtrada, editada linha a linha e exportada. Em seguida, o usuário pede "marque o lançamento 12 como conciliado". O sistema não grava nada ainda: abre um card de confirmação com o formulário já preenchido, o usuário confere, clica em **Revisar**, e só depois de **segurar o botão Confirmar por 2 segundos** a alteração é gravada e registrada na auditoria.

2.2 Fundamentação teórica

Jakob Nielsen descreveu a IA generativa como a primeira mudança de paradigma de interface em 60 anos: a transição da manipulação direta (o usuário especifica como fazer) para a especificação de intenção (o usuário declara o que quer, e o sistema resolve o como).

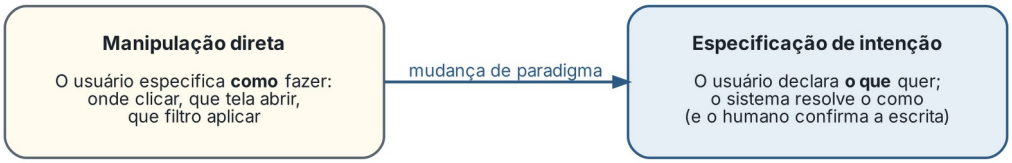


Figura 1 — Mudança de paradigma de interface (Nielsen, 2023).

O mecanismo técnico que viabiliza essa transição é o **tool calling** (chamada de ferramentas): o modelo de linguagem recebe uma lista de ferramentas disponíveis e decide qual invocar e com quais argumentos, mas **não executa** a operação. O padrão de raciocínio e ação foi formalizado no artigo ReAct (Yao et al., 2022) e é a base da documentação oficial de tool use dos principais provedores. O modelo de supervisão adotado, em que toda ação de escrita depende de confirmação humana, é conhecido como **human-in-the-loop**. A equipe resume o argumento como "um Jarvis com freio": o agente entende, sugere e prepara; quem aperta o botão é o humano. Numa empresa de contabilidade, um erro do modelo vira multa, e por isso a confirmação é o diferencial do sistema, não uma limitação aceita.

2.3 Os quatro padrões de interação

Tabela 2 — Os quatro padrões de interação e sua situação nesta versão

Padrão	Como funciona no Sentinel	Situação
Consulta com resultado tabular	O usuário pergunta; o agente escolhe uma tool `consultar_*`; o servidor executa e devolve uma tabela mascarada, com botão Editar por linha e botão Exportar (CSV).	Implementado

Padrão	Como funciona no Sentinel	Situação
Cadastro ou atualização por formulário pré-preenchido	O agente escolhe `criar_*` ou `atualizar_*`; o card abre o formulário com o que a IA entendeu; o humano edita, revisa e segura para confirmar. A validação é do backend (Form Request).	Implementado
Ação em massa com confirmação	O agente apresenta um **plano** com a lista das ações e o impacto; o humano revisa o plano inteiro e confirma uma vez; a gravação é tudo ou nada.	Especificado (seção 13.3)
Log de auditoria	Toda decisão (sucesso, erro, recusa, negação, cancelamento, expiração) vira uma linha em `audit_logs`, consultável por administradores, com o que a IA propôs e o que o humano confirmou.	Implementado

3. Público-alvo e mercado

A definição do público-alvo segue o mesmo rigor exigido do restante do projeto e se apoia em dados oficiais e recentes (Sebrae, 2025).

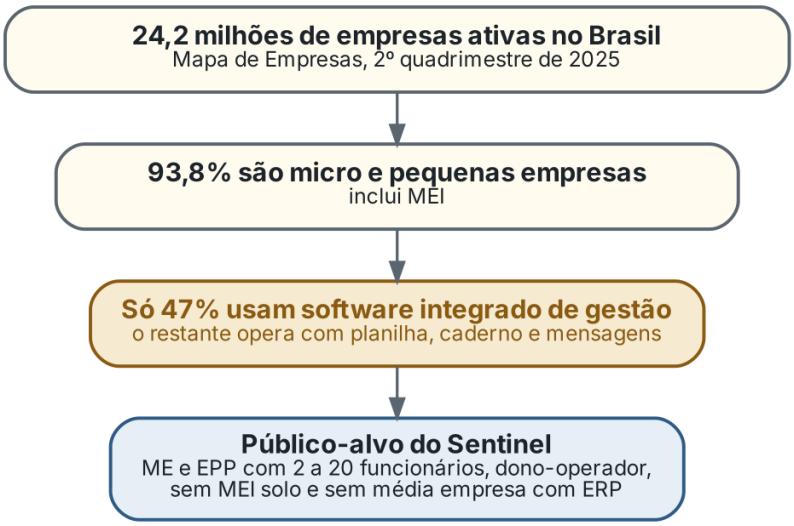


Figura 2 — Funil de definição do público-alvo com base em dados do Sebrae.

3.1 O tamanho e a natureza do mercado

O Brasil encerrou o segundo quadrimestre de 2025 com 24,2 milhões de empresas ativas, das quais 93,8% são micro e pequenas empresas. Apenas 47% dos pequenos negócios utilizam softwares ou aplicativos integrados de gestão.

3.2 O gargalo é a interface, não o acesso

76% dos empreendedores já usam computador nas atividades, e 88% usam internet no dia a dia, concentrada em redes sociais e serviços bancários; menos de 13% usam plataformas de gestão. O padrão é o de um empreendedor fluente em interface de conversa e pouco familiarizado com interface de sistema, exatamente a lacuna que a interface conversacional agêntica ataca.

3.3 Definição do público-alvo

Micro e pequenas empresas brasileiras com aproximadamente 2 a 20 funcionários, geridas por um dono-operador sem departamento administrativo dedicado, excluindo o MEI que trabalha sozinho e a média empresa já madura em ERP. No domínio escolhido, o usuário típico é o contador ou o assistente de um escritório que atende essas empresas.

> **Nota metodológica**
>
> As seções 3.1 a 3.3 apresentam estatísticas de fontes oficiais. A seção 3.4 apresenta inferências e projeções da equipe construídas sobre esses dados: hipóteses argumentadas, não fatos estabelecidos.

3.4 Predições sobre o público-alvo

O público cresce por substituição geracional

Entre empreendedores de até 34 anos, o uso de aplicativos de gestão já chega a 57%, contra 47% na média geral. A janela de encaixe do produto não se fecha: abre-se progressivamente.

O mercado tende a pular a era do ERP tradicional

Os 53% de pequenos negócios que nunca adotaram software de gestão dificilmente passarão por uma fase intermediária de ERP de menus. O concorrente direto não é um ERP estabelecido: é a planilha e a conversa informal.

A adoção será decidida por confiança, não por capacidade técnica

A retenção deve se correlacionar mais com a qualidade do fluxo de confirmação do que com a quantidade de funcionalidades. Por isso o card de confirmação (Revisar → segurar) recebeu a maior parte do esforço de engenharia e de testes do projeto.

O uso tenderá a ser móvel, fragmentado e falado

Um público que já administra fornecedores e clientes por mensagem de voz tende a tratar a entrada por voz como expectativa natural. A interface foi construída responsiva (360 px sem rolagem horizontal); a voz fica como trabalho futuro.

A interface estrutura o dado que hoje não existe

Cada intenção do usuário vira uma chamada de ferramenta tipada e auditada. O sistema estrutura o dado como efeito colateral do uso comum, o que reabre, no futuro, a camada preditiva descartada no início.

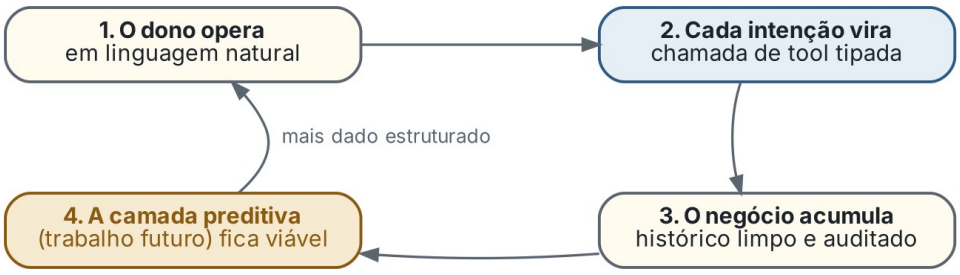


Figura 3 — Ciclo pelo qual a interface conversacional viabiliza, no futuro, a camada preditiva do anteprojeto.

Risco previsto: fadiga de confirmação

É esperado que o usuário passe a confirmar sem revisar. O sistema responde com fricção proporcional ao risco: a etapa **Revisar** obriga a olhar o formulário antes de o botão de segurar existir; a borda do card muda de cor conforme o risco (escrita ou exclusão); e restaurar ou excluir de vez exige a senha de um administrador.

Síntese

O público-alvo não se define apenas pelo porte da empresa, mas por um perfil cognitivo: gestores cuja alfabetização digital ocorreu em interfaces de conversa, administrando negócios complexos demais para a planilha e simples demais para um ERP.

4. Visão geral do sistema construído

Esta seção resume o que existe no repositório ao fim da construção (Fases 0 a 5), na revisão `ae79781` da branch `main`.

Tabela 3 — O sistema em números

Indicador	Valor	Observação
Tools (ferramentas) no catálogo do agente	37	7 consultas, 6 cadastros, 6 atualizações, 6 exclusões, 6 restaurações, 6 exclusões definitivas
Entidades de domínio	6	Lançamento, categoria de lançamento, cliente, fornecedor, funcionário, nota fiscal
Papéis de acesso	3	leitura < operador < admin
Migrations	15	MySQL em desenvolvimento; compiladas também para PostgreSQL
Testes automatizados	398 casos	29 arquivos, 3.882 asserções, todos verdes
Rotas autenticadas	15	Todas atrás de `auth` e CSRF; 14 também passam pela varredura de propostas vencidas
Código de aplicação (app/)	≈ 6.500 linhas	O `AiAgentService` sozinho tem ≈ 1.950 linhas

4.1 Stack tecnológica

Tabela 4 — Stack tecnológica

Camada	Tecnologia	Por quê
Backend	Laravel 13 sobre PHP 8.4	Framework MVC maduro, com Form Requests, Eloquent, rate limiting e autenticação nativos; regra do projeto: 100% PHP
Interface	Blade, Tailwind CSS 4 (Vite), JavaScript puro	Sem framework de front-end: o HTML é sempre escrito pela equipe, nunca gerado pela IA
Banco de dados	MySQL (Laragon) em desenvolvimento e na demonstração	A rede do SENAI bloqueia o protocolo PostgreSQL; a migração para Supabase (PostgreSQL, região de São Paulo) fica para quando o projeto sair de protótipo
IA	Microsoft Foundry, Chat Completions API com `tools`/`tool_choice`	Chamada HTTP REST única e stateless, sem SDK e sem Agent Service (threads/runs): menos complexidade e menos exposição a uma API em transição
Testes	PHPUnit, SQLite em memória, provedor de IA simulado (`Http::fake`)	A suíte inteira roda offline, sem custo de API e de forma determinística
Vitrine	Three.js (apenas na página pública, antes do login)	Farol 3D que gira com a rolagem; a área de trabalho não tem animação

4.2 Papéis e permissões

O controle de acesso é **por ferramenta**, não só por tela: cada tool declara um papel mínimo, e a hierarquia é `leitura < operador < admin`. Papel desconhecido ou ausente nega tudo (falha fechada). O papel e o tenant (a empresa) vêm sempre do usuário logado.

Tabela 5 — Matriz de permissões (RBAC por ferramenta)

Ação	leitura	operador	admin
Consultar lançamentos, categorias, clientes, fornecedores, notas e funcionários	Sim	Sim	Sim
Ver salário (em R$) no card de funcionário	Não	Não	Sim (só no card)
Criar e atualizar lançamento, categoria, cliente, fornecedor e nota fiscal	Não	Sim	Sim
Criar e atualizar funcionário	Não	Não	Sim
Consultar a lixeira	Não	Sim	Sim
Excluir (mover para a lixeira)	Não	Não	Sim
Restaurar da lixeira (com senha de um admin)	Não	Sim (funcionário: só admin)	Sim
Excluir de vez um registro arquivado (com senha)	Não	Não	Sim
Consultar a auditoria	Não	Não	Sim
Exportar CSV de uma consulta	Sim	Sim	Sim

4.3 Telas

Tabela 6 — Telas do sistema

Tela	Quem vê	Para que serve
`/` (vitrine)	Público	Apresentação do produto; único botão leva ao login
`/login`	Público	Login por sessão; 5 tentativas por minuto por e-mail + IP
`/agente`	Todos os papéis	Conversa com o agente, tabelas de consulta e card de confirmação
`/cadastros`	Todos os papéis	Modo manual sem IA: listar, buscar, cadastrar, editar e exportar
`/lixeira`	operador e admin	Itens na lixeira e arquivados; restaurar e excluir de vez
`/auditoria`	admin	Histórico filtrável e paginado; detalhe com proposto × confirmado
Telas de erro	Todos	401, 403, 404, 419, 429, 500 e 503 em português, com o que fazer

5. Arquitetura

O princípio arquitetural inegociável do projeto é que **o modelo de linguagem nunca toca o banco de dados**. O LLM é uma camada de decisão: escolhe, entre ferramentas pré-registradas pelo backend, qual invocar e com quais argumentos. Quem executa (valida, grava, audita) é o próprio Laravel, de forma inteiramente determinística. A mesma regra vale para a tela: a IA nunca gera HTML; ela devolve dados, e a interface monta componentes que a equipe escreveu.

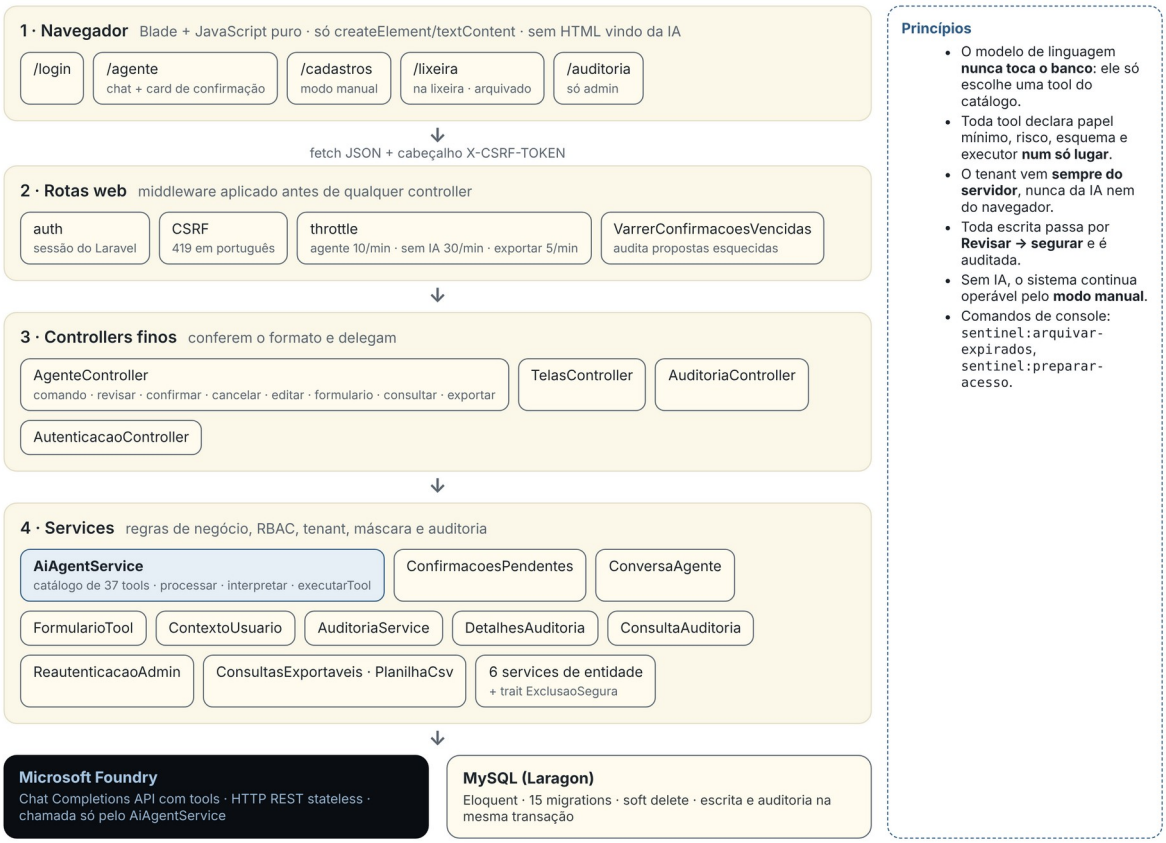


Figura 4 — Arquitetura em camadas do Sentinel e os princípios que a regem.

5.1 As camadas

Tabela 7 — Camadas e responsabilidades

Camada	Responsabilidade	Determinística?
Intenção (LLM no Microsoft Foundry)	Interpretar a linguagem natural e escolher uma tool do catálogo, com argumentos	Não: é a única camada não determinística, e por isso tudo o que sai dela é tratado como dado não confiável
Orquestração (`AiAgentService`)	Montar o prompt, o histórico e a lista de tools; validar a resposta do modelo contra o catálogo e o esquema; decidir se executa (leitura) ou propõe (escrita)	Sim

Camada	Responsabilidade	Determinística?
Ferramentas (catálogo de 37 tools)	Cada tool declara papel mínimo, risco, esquema JSON, regras de validação e executor	Sim
Execução (services, Form Requests, Eloquent)	Regras de negócio, validação, tenant, transação e auditoria	Sim
Apresentação (Blade + JS)	Montar tabela, formulário, card e mensagens a partir do JSON; só `textContent`	Sim

5.2 Componentes principais

Tabela 8 — Componentes e responsabilidades

Classe	Papel no sistema
`AiAgentService`	Única porta para a IA. Contém o catálogo de tools (fonte única: o que não está nele não é oferecido nem executável), `processar`/`interpretar`, `executarTool` (RBAC + tenant + transação + auditoria), `revisar`, `abrirFormulario`, `consultarManual`, `exportar` e `auditarDesfecho`
`ConfirmacoesPendentes`	Estado das propostas na sessão: rascunho (5 min), token de segurar (válido de 2 a 12 s), uso único atômico (`Cache::add`), varredura de vencidas e tempo da IA
`FormularioTool`	Converte as regras do Form Request nos campos do formulário do card (tipo, obrigatório, opções, condições)
`ConversaAgente`	Memória curta da conversa: até 16 mensagens e 8.000 caracteres, só texto, isolada por usuário e tenant, higienizada
`ContextoUsuario`	Usuário, papel e tenant do usuário logado; lança exceção se faltar qualquer um (falha fechada)
`AuditoriaService`	Ponto único de gravação em `audit_logs` e fonte única da máscara LGPD (CPF, salário, documento)
`DetalhesAuditoria`	Escopo por requisição para campos extras da auditoria: origem, proposto, quem autorizou, tempo da IA, valores anteriores
`ConsultaAuditoria`	Leitura da auditoria: só admin (a negação também é auditada), por tenant, filtros, 25 por página, detalhe mascarado
`ReautenticacaoAdmin`	Confere a senha de um admin do mesmo tenant, com limites atômicos de tentativas
`ConsultasExportaveis` e `PlanilhaCsv`	Guardam a consulta na sessão (o navegador só conhece um id) e geram o CSV seguro
`ExclusaoSegura` (trait)	Excluir com bloqueio por dependência, restaurar, excluir de vez; motivos em linguagem simples
Services de entidade	`LancamentoService`, `CategoriaLancamentoService`, `ClienteService`, `FornecedorService`, `FuncionarioService`, `NotaFiscalService`, todos com `int $tenantId` obrigatório

Classe	Papel no sistema
`AgenteController`	Rotas do agente: comando, revisar, confirmar, editar, cancelar, formulário, consultar e exportar
`TelasController`, `AuditoriaController`, `AutenticacaoController`	Telas de Cadastros, Lixeira e Auditoria; JSON da auditoria; login e logout (o logout audita as propostas abertas)
`VarrerConfirmacoesVencidas` (middleware)	Depois de cada requisição autenticada, audita como "expirado" as propostas esquecidas
Comandos Artisan	`sentinel:arquivar-expirados` (RF11, agendado diariamente) e `sentinel:preparar-acesso` (usuários locais)

5.3 Rotas

Tabela 9 — Rotas autenticadas e seus limites

Método e rota	Limite	Função
POST `/login`	5/min por e-mail + IP	Autenticação por sessão; regenera a sessão
POST `/agente/comando`	10/min por usuário	Envia a mensagem ao agente (única rota que chama a IA)
POST `/agente/revisar`	30/min	Valida o formulário editado sem gravar e emite o token de segurar
POST `/agente/confirmar`	uso único do token	Executa a proposta revisada (valores vindos da sessão)
POST `/agente/cancelar` e `/agente/editar`	30/min	Cancela (auditado) ou volta ao formulário (anula o token)
POST `/agente/formulario`	30/min	Abre o formulário de criar ou atualizar sem IA
POST `/agente/consultar`	30/min	Modo manual: consulta sem IA
POST `/agente/exportar`	5/min	Gera o CSV de uma consulta, refeita no servidor
GET `/auditoria/registros[/{id}]`	60/min	Lista e detalhe da auditoria em JSON (só admin)
GET `/agente`, `/cadastros`, `/lixeira`, `/auditoria`	—	Telas (os dados vêm das rotas JSON)

5.4 Decisões arquiteturais

Tabela 10 — Decisões de arquitetura e alternativas avaliadas

Decisão	Alternativa descartada	Motivo
Chat Completions API, chamada stateless	Agent Service / Assistants API (threads e runs)	Menor complexidade no prazo do TCC e menor exposição a uma API em transição. A confirmação humana é implementada pela equipe, não por recurso do provedor. Toda a integração fica em uma classe: trocar de provedor é mexer em um ponto só
Catálogo estático de tools (whitelist)	MCP para as ferramentas internas	MCP favorece a descoberta dinâmica de ferramentas, o oposto da whitelist que sustenta a governança. Fica como candidato para operar serviços externos no futuro
Engenharia de prompt + tool calling	Fine-tuning de modelo	Exigiria dados rotulados e tempo de treino incompatíveis com o prazo; o tool calling já entrega a precisão necessária
Consultas estruturadas ao banco	RAG (busca semântica)	As operações centrais são consultas estruturadas, não busca em texto. RAG fica como extensão possível para um modo de ajuda
Catálogo fechado de componentes de tela	IA gerando HTML	Previsibilidade visual, auditabilidade, acessibilidade e eliminação de XSS vindo do modelo
MySQL local	Supabase PostgreSQL já no TCC	A rede do SENAI descarta o protocolo PostgreSQL. As migrations já foram compiladas para PostgreSQL; o checklist de migração está no `CLAUDE.md`

5.5 Resiliência quando a IA falha

O cenário mais frequente não é a falta total de internet, mas a API de IA lenta, fora do ar ou recusando a chamada. Nesse caso o Sentinel **degrada sem travar**:

- a chamada tem timeout de 30 s no servidor (o navegador desiste em 35 s), e toda falha vira uma mensagem amigável no contrato de erro, com um `correlation_id` para localizar o log;
- o **modo manual** (`/cadastros` e `/lixeira`) faz todo o CRUD sem chamar a IA: listar, buscar, cadastrar, editar, exportar, restaurar e excluir de vez;
- a governança é a mesma nos dois caminhos: o modo manual usa o mesmo `executarTool()` e o mesmo fluxo Revisar → segurar, e a auditoria registra a origem (`agente`, `formulario` ou `manual`).

A fila local de comandos prevista na versão 1.0, que guardaria a intenção para reapresentá-la quando a IA voltasse, **não foi implementada** nesta versão: o modo manual cobre o requisito de continuidade (RNF05) com menos complexidade. A ausência total de conectividade (PWA ou aplicativo com banco local) continua como trabalho futuro.

6. O agente

O agente é o conjunto formado pelo modelo de linguagem no Microsoft Foundry e pela camada de orquestração (`AiAgentService`). O modelo decide; o servidor confere tudo o que o modelo decidiu antes de agir.

6.1 O catálogo de ferramentas

O catálogo é montado por quatro construtores (`definirTool`, `definirEscrita`, `definirAtualizacao` e `definirLixeira`) e é a **fonte única** do que existe: a lista enviada à IA, o RBAC, o formulário do card, as colunas da tabela e o executor saem da mesma definição. Uma tool que não está no catálogo não é oferecida ao modelo e, se o modelo a inventar, não é executada.

Tabela 11 — Catálogo de tools do agente

Grupo	Tools	Risco	Papel mínimo
Consultas (7)	`consultar_lancamentos`, `consultar_categorias_lancamento`, `consultar_clientes`, `consultar_fornecedores`, `consultar_funcionarios`, `consultar_notas_fiscais`, `consultar_lixeira`	leitura	leitura (lixeira: operador)
Cadastros (6)	`criar_lancamento`, `criar_categoria_lancamento`, `criar_cliente`, `criar_fornecedor`, `criar_funcionario`, `criar_nota_fiscal`	escrita	operador (funcionário: admin)
Atualizações (6)	`atualizar_status_lancamento`, `atualizar_categoria_lancamento`, `atualizar_cliente`, `atualizar_fornecedor`, `atualizar_funcionario`, `atualizar_nota_fiscal`	escrita	operador (funcionário: admin)
Exclusões (6)	`excluir_<entidade>`: move para a lixeira se nada depender do registro	exclusão	admin
Restaurações (6)	`restaurar_<entidade>`: tira da lixeira; exige senha de um admin no card	escrita	operador (funcionário: admin)
Exclusões definitivas (6)	`excluir_definitivo_<entidade>`: só de registro arquivado; exige senha de admin	exclusão	admin

6.2 O que vai e o que nunca vai para a IA

A fronteira de dados com o provedor de IA é tratada como fronteira de segurança. O modelo recebe o mínimo para decidir **qual** ferramenta usar; ele nunca vê os dados que a ferramenta devolve.

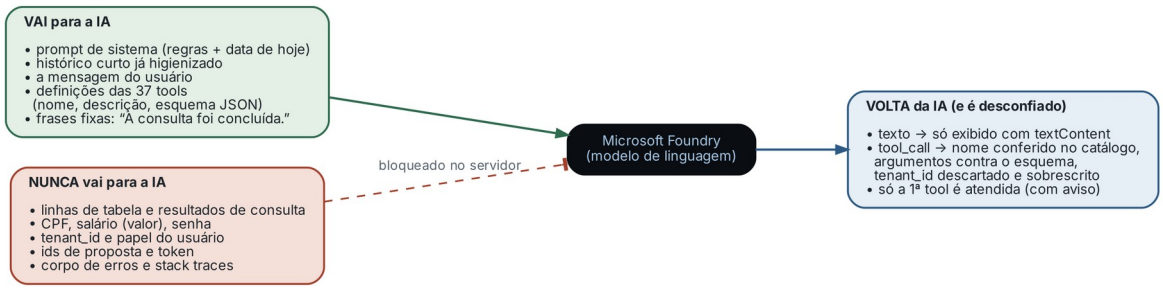


Figura 5 — Fronteira de dados entre o Sentinel e o provedor de IA.

O prompt de sistema fixa o papel do assistente (contabilidade de micro e pequenas empresas), a data de hoje (para resolver "ontem" e "hoje") e as regras de conduta: usar só as ferramentas disponíveis, não inventar ferramentas nem dados, perguntar quando faltar um campo obrigatório, não pedir confirmação em texto (a tela já faz isso), lembrar que proposta não é escrita executada, e **nunca pedir nem aceitar senha no chat**.

6.3 Memória da conversa

`ConversaAgente` guarda na sessão as últimas 16 mensagens (no máximo 8.000 caracteres), só texto, ligadas ao usuário e ao tenant: se um deles mudar, a memória é apagada, e o logout a limpa. Antes de guardar, o texto é higienizado: mensagens com termos de credencial são omitidas, CPF e CNPJ viram "[documento omitido]" e o valor que vem depois de "salário" ou "remuneração" vira "[valor omitido]". Resultados de consulta e propostas entram no histórico apenas como frases fixas ("A consulta foi concluída"), sem nenhum dado.

6.4 Como a resposta do modelo é conferida

1. A resposta HTTP precisa ser bem-sucedida; caso contrário, vira erro classificado (tabela abaixo).
2. A mensagem precisa ter o formato esperado (objeto, `tool_calls` em lista, `finish_reason` coerente). Resposta truncada ou filtrada pelo provedor vira erro próprio.
3. Se for só texto, é exibida como texto (no navegador, sempre por `textContent`). O log guarda só o tempo, nunca o conteúdo.
4. Se houver chamadas de tool, **todas** são validadas antes de executar a primeira: JSON válido, objeto, e nenhuma chave fora do esquema da tool. `tenant_id` é sempre removido.
5. Só a primeira chamada é atendida; se havia mais, a resposta traz um aviso pedindo que o usuário peça uma de cada vez.
6. Tool de leitura é executada na hora (com RBAC, tenant e auditoria). Tool de escrita passa pelo RBAC e vira **proposta**: nada é gravado nesse momento.

Tabela 12 — Contrato de erro do agente (todas as respostas levam também um correlation_id)

Situação	Categoria	Código	HTTP
Endpoint, chave ou deployment não configurado	`provider_configuration_error`	`AI_PROVIDER_UNAVAILABLE`	503

Situação	Categoria	Código	HTTP
Falha de conexão	`provider_connection_error`	`AI_PROVIDER_UNAVAILABLE`	503
Tempo esgotado (30 s)	`provider_timeout`	`AI_PROVIDER_TIMEOUT`	504
Provedor recusou a credencial (401/403)	`provider_authentication_error` / `..._authorization_error`	`AI_PROVIDER_UNAVAILABLE`	502
Limite do provedor (429) ou erro 5xx	`provider_rate_limit` / `provider_http_error`	`AI_PROVIDER_UNAVAILABLE`	503
Resposta fora do formato	`provider_response_error`	`AI_PROVIDER_RESPONSE_INVALID`	502
Resposta truncada / filtrada	`provider_response_error`	`AI_RESPONSE_TRUNCATED` / `AI_CONTENT_FILTERED`	502
Falha técnica ao executar uma tool	`tool_error`	`AI_TOOL_ERROR`	500

6.5 Tempo de resposta

A chamada ao provedor é cronometrada com `hrtime` e o tempo é gravado em `audit_logs.tempo_ia_ms` na linha do desfecho de cada pedido; o detalhe da auditoria mostra "Tempo da IA". Isso torna o RNF04 **mensurável**. O limite de 10 s, porém, **não é imposto** pelo código (o teto técnico é o timeout de 30 s): a decisão sobre o limite ficou em aberto e será tomada com os números medidos no teste manual (seção 13).

7. Fluxos da aplicação

Os fluxogramas desta seção descrevem o comportamento real do código. As cores seguem a sinalização da interface: âmbar para escrita pendente, verde para sucesso, vermelho para negação, recusa, cancelamento ou expiração, azul para decisões e fundo escuro para a IA.

7.1 F1: do comando à gravação

É o fluxo central do sistema. Toda escrita, venha do agente ou do modo manual, termina nas mesmas etapas de **Revisar** e **segurar**.

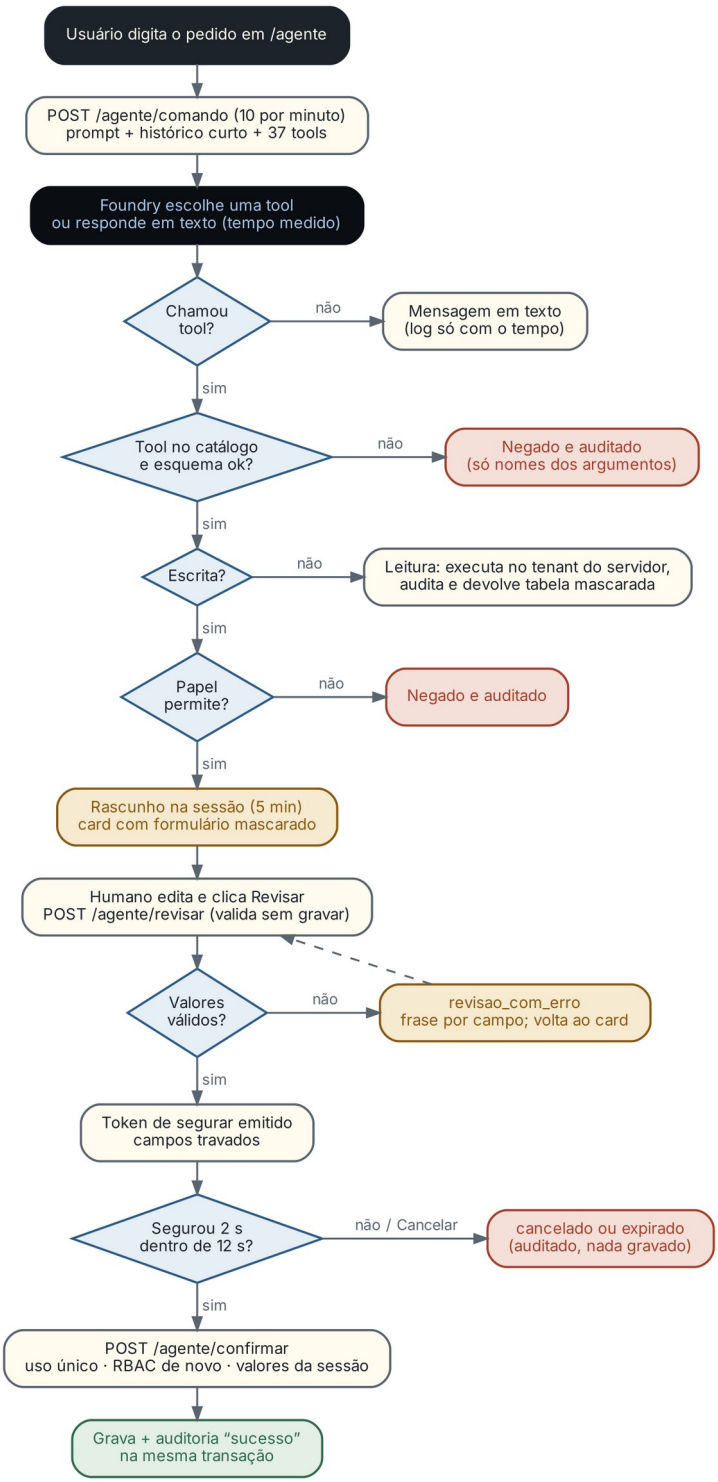


Figura 6 — F1: comando, proposta, Revisar, segurar e confirmar.

1. O usuário digita o pedido em `/agente`, que chega a `POST /agente/comando` (limite de 10 por minuto).
2. `AiAgentService::processar()` monta o prompt de sistema, o histórico curto e o catálogo de tools, e chama a Chat Completions API. O tempo da chamada é medido.
3. `interpretar()` confere a resposta (seção 6.4). Tool fora do catálogo ou com argumento fora do esquema é negada e auditada só com os **nomes** dos argumentos.

4. Tool de escrita: o RBAC confere o papel. Sem permissão, "negado" (auditado). Com permissão, a resposta é `confirmacao_pendente` com o formulário. O controller guarda a proposta crua na sessão como **rascunho** (5 minutos) e devolve ao navegador só os valores mascarados e um id de proposta.
5. O humano edita o formulário e clica em **Revisar** (`POST /agente/revisar`): RBAC, tenant e regras da tool são aplicados **sem gravar**. Com erro, o formulário volta com a frase de cada campo (auditado como `revisao_com_erro`, que não é desfecho). Sem erro, sai o **token** de segurar e os campos travam.
6. O humano **segura** Confirmar por 2 s, e o navegador envia `POST /agente/confirmar` com o token. O servidor exige uso único, espera mínima de 2 s e validade de 12 s; a tool e os valores vêm da sessão, nunca do corpo da requisição.
7. `executarTool()` refaz o RBAC, injeta o tenant do servidor e executa a tool e a auditoria "sucesso" **na mesma transação**.
8. Cancelar é auditado como "cancelado" ou "expirado" conforme o horário. Proposta abandonada é auditada como "expirado" pela varredura na requisição seguinte; no logout, as propostas abertas também recebem desfecho.

7.2 Estados de uma proposta

Cada proposta de escrita é uma pequena máquina de estados guardada na sessão. Ela termina em **exatamente um** desfecho, e cada desfecho é uma linha na auditoria.

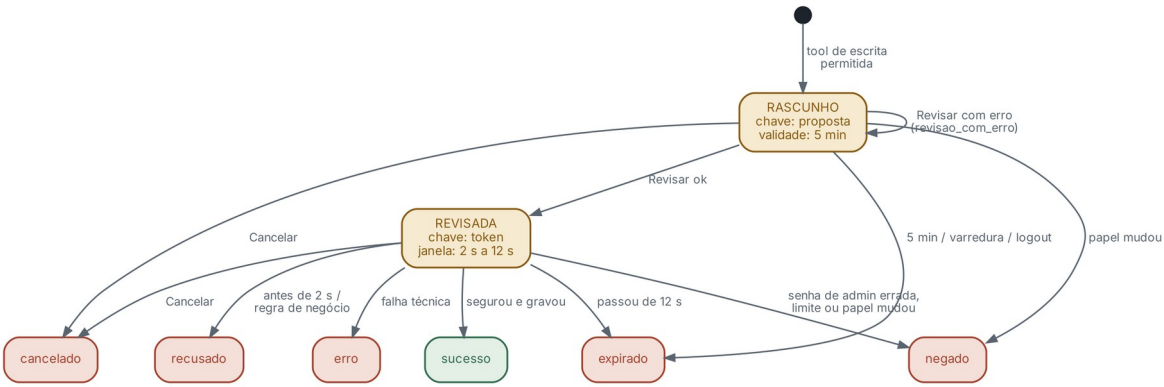


Figura 7 — Máquina de estados da proposta de escrita.

Tabela 13 — Resultados registrados na auditoria

Resultado	Final?	Quando acontece
`sucesso`	Sim	A escrita foi gravada (na mesma transação da auditoria)
`negado`	Sim	Falta permissão (papel), ou a senha de admin não conferiu, ou o limite de tentativas estourou
`recusado`	Sim	Regra de negócio (ex.: dependência, alteração concorrente) ou confirmação antes da espera mínima de 2 s
`cancelado`	Sim	O humano clicou em Cancelar dentro do prazo
`expirado`	Sim	Passou do prazo (12 s do token ou 5 min do rascunho), por cancelamento tardio, varredura ou logout
`erro`	Sim	Falha técnica durante a execução; nada é gravado

Resultado	Final?	Quando acontece
`revisao_com_erro`	Não	O Revisar encontrou erro de validação; a proposta continua em rascunho

7.3 Segurar para confirmar

A confirmação usa o padrão **hold-to-confirm**: o botão Confirmar precisa ser pressionado e mantido por 2.000 ms. Enquanto ele é mantido, o preenchimento avança de 0% a 100% e o texto mostra "confirmando" e a porcentagem. Soltar antes do fim zera o botão e mostra "Você soltou antes do fim. Nada foi gravado.". O gesto funciona com mouse, toque e teclado (Enter ou Espaço mantidos).

- **Duas janelas de tempo:** depois do Revisar, a interface dá 10 s para segurar; o servidor aceita até 12 s (folga para a rede). Antes disso existe o prazo absoluto de 5 min do rascunho, que não reinicia ao editar.
- **O card não some calado:** quando o prazo passa, ele fica visível com "Confirmação expirada. Nada foi gravado. Peça de novo." e os botões desativados.
- **Dois canais sempre:** cor da borda (risco) + título + texto da etapa + anúncio para leitor de tela (`aria-live`) + popup com símbolo e texto ao final (RNF07).
- **Cancelar é neutro** (não vermelho) e cancela com um clique; o vermelho fica para a ação destrutiva.

7.4 F2: atualizar um registro

A atualização começa pelo agente ("mude o telefone do cliente X") ou pelo botão **Editar** de uma linha de tabela, que abre o formulário sem chamar a IA. O servidor localiza o registro no tenant e protege contra alteração concorrente usando a data de atualização como versão.

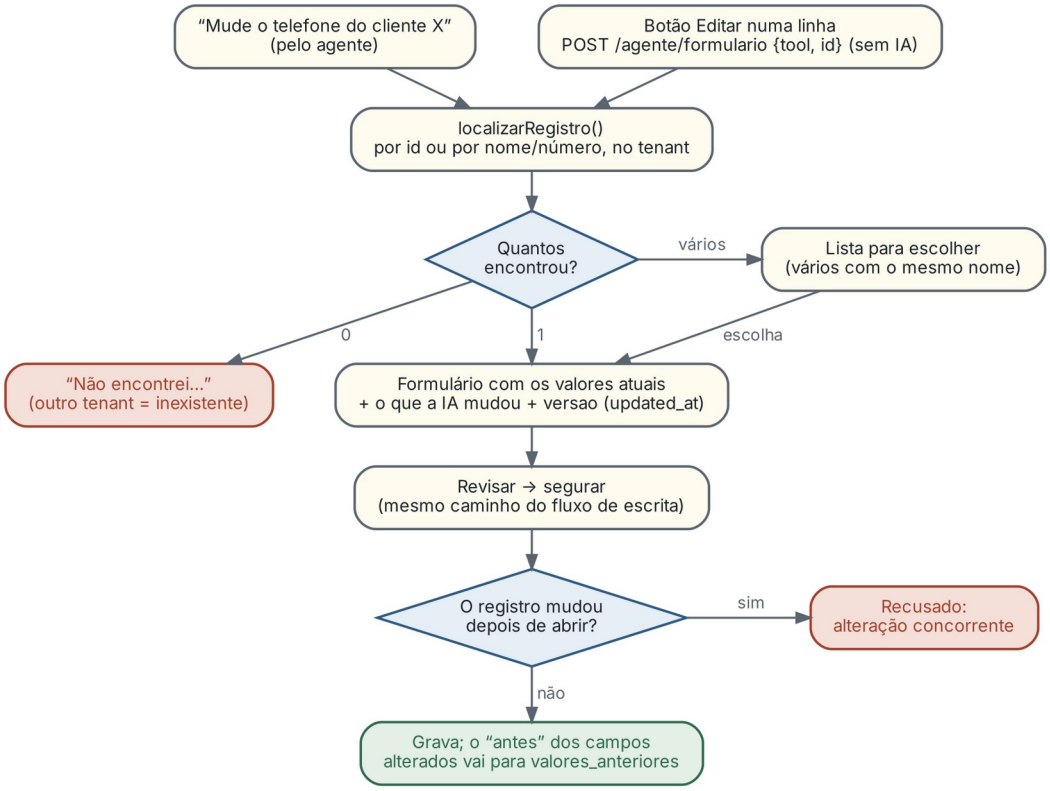


Figura 8 — F2: localizar o registro, abrir o formulário e gravar com controle de versão.

7.5 F3: lixeira, arquivamento e exclusão definitiva

Nenhum dado se perde por engano. A exclusão é um ciclo de vida com quatro estados, e só a última transição é irreversível, sempre por decisão manual de um administrador.

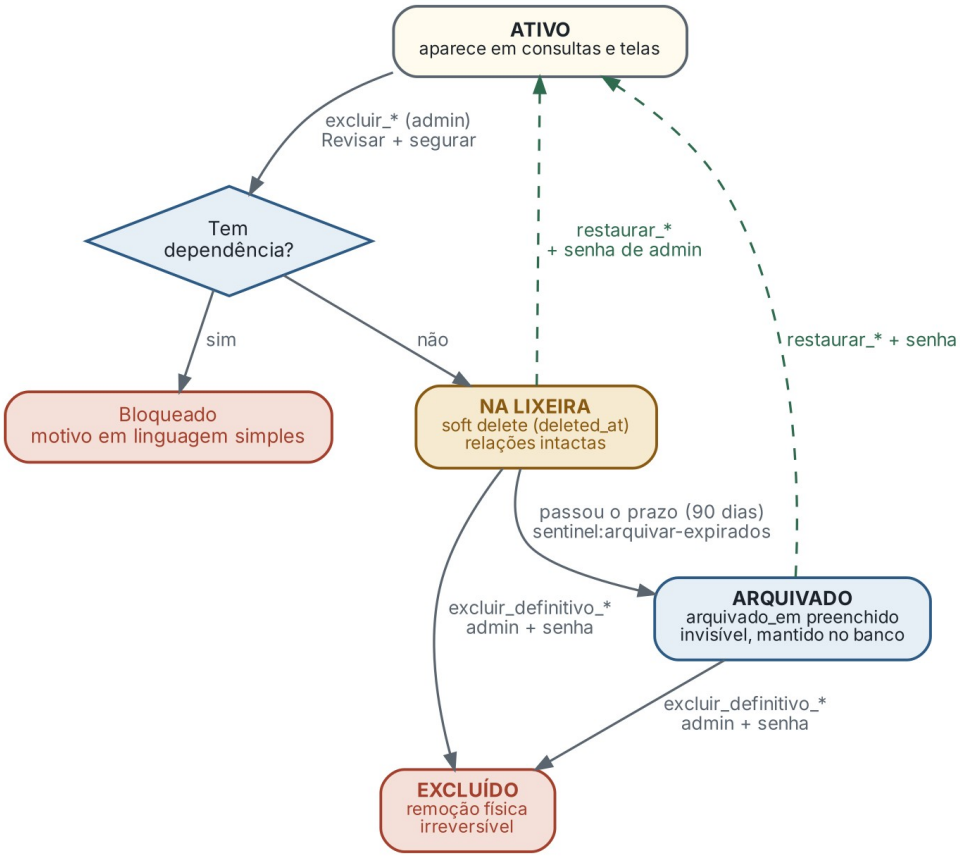


Figura 9 — Ciclo de vida de um registro: ativo, lixeira, arquivado e excluído.

- **Bloqueio por dependência (RF08):** antes de excluir, o sistema verifica as relações que impedem a exclusão e explica o motivo em linguagem simples ("Não é possível excluir: existem lançamentos vinculados a este registro."). A recusa aparece já no Revisar e é conferida de novo no segurar.
- **Lixeira (RF09):** soft delete; as relações continuam intactas. Prazo de retenção configurável (`SENTINEL_RETENCAO_LIXEIRA_DIAS`, padrão 90 dias).
- **Arquivamento (RF11):** passado o prazo, o comando agendado `sentinel:arquivar-expirados` marca o registro como arquivado. Ele some das telas, mas continua no banco, porque documentos fiscais e trabalhistas têm prazo legal de guarda de anos.
- **Restauração (RF10):** volta o registro com a data de cadastro e as relações originais; exige a senha de um admin do mesmo tenant, e a auditoria grava quem autorizou (`autorizado_por`). Restaurar um registro cujo "pai" está na lixeira é recusado, dizendo qual é o pai.
- **Exclusão definitiva (RF12):** só de registro arquivado, só por admin e com senha. Nenhuma rotina automática faz exclusão física.

A ordem das verificações na confirmação com senha foi desenhada para que o card **não sirva de verificador de senhas**: tudo o que não envolve segredo é conferido primeiro, e um usuário sem permissão recebe a mesma resposta com senha certa ou errada, sem gastar tentativa.

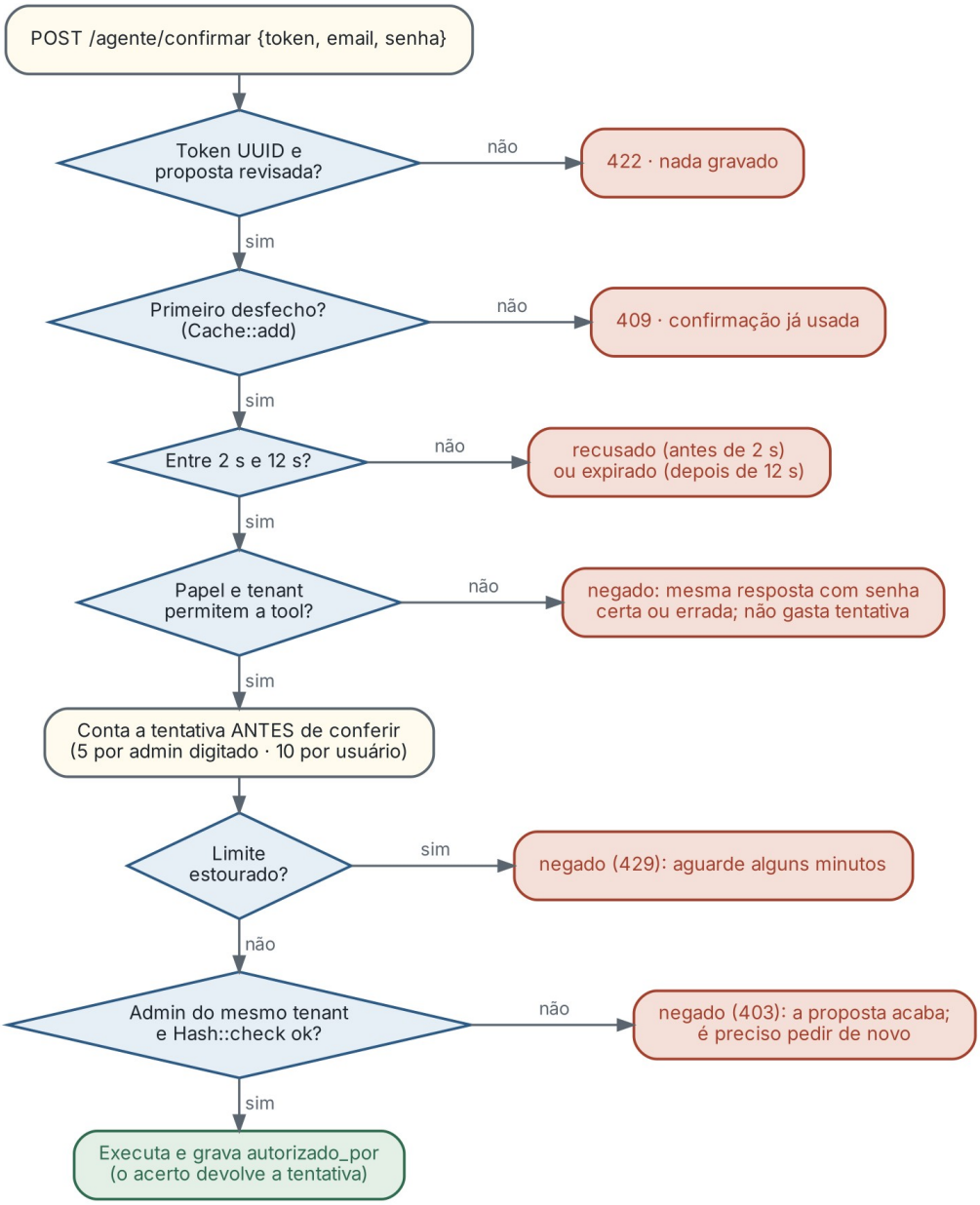


Figura 10 — Ordem das verificações em uma confirmação que exige senha de administrador.

7.6 F4: consultar e exportar

O navegador nunca diz ao servidor **o que** exportar: ele envia apenas um id. A consulta (tool e argumentos) fica guardada na sessão, e na exportação o servidor a refaz com o RBAC, o tenant e a máscara do momento. Assim, um usuário que perdeu permissão depois da consulta não consegue exportar, e um id adulterado não leva a dados de outro tenant.

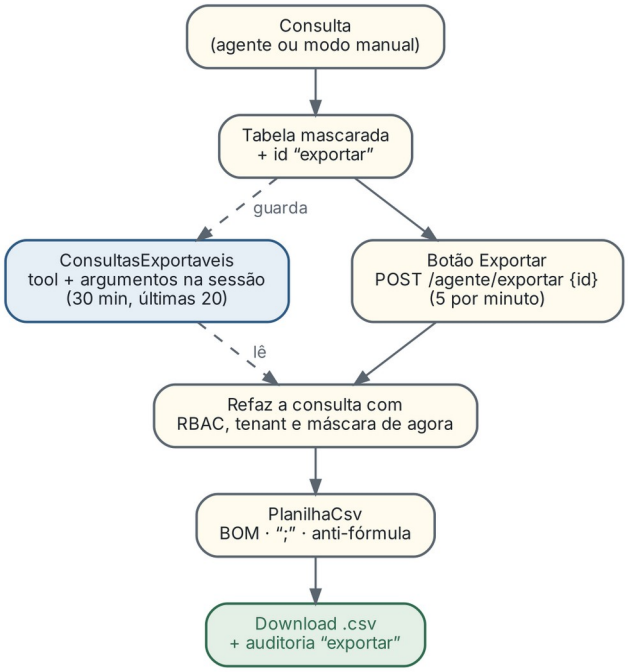


Figura 11 — F4: consulta, guarda da consulta na sessão e exportação segura em CSV.

7.7 F5: modo manual, sem IA

As telas Cadastros e Lixeira funcionam sem o agente. Elas chamam `POST /agente/consultar` com a tool e os filtros; o servidor aceita só tools de leitura do catálogo e confere os filtros contra o esquema da tool. "Novo" e "Editar" abrem o mesmo card de confirmação, sem IA. É assim que o sistema atende ao RNF05 quando a Azure está fora do ar.

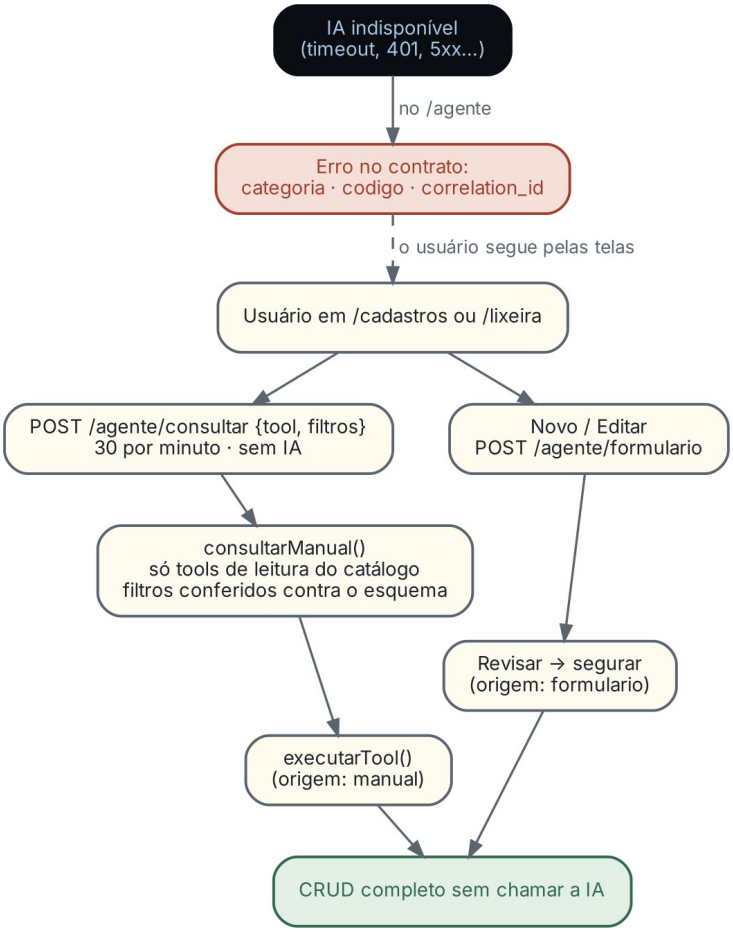


Figura 12 — F5: modo manual e degradação quando a IA está indisponível.

8. Segurança

O Sentinel é um protótipo acadêmico, mas trata dados que seriam sensíveis em uma empresa real: CPF, salário e movimentação financeira. A regra do projeto é que **controle de escopo não é desculpa para segurança relapsa**: não se adiciona infraestrutura de produção (filas, 2FA, observabilidade), mas tudo o que está no escopo é feito com cuidado e coberto por teste.

8.1 Princípios

- **A IA é tratada como entrada não confiável.** O que vem do modelo passa pelas mesmas verificações que um formulário vindo de um usuário mal-intencionado.
- **Defesa em profundidade.** O RBAC é conferido na proposta, no Revisar e de novo na confirmação; o tenant é injetado pelo servidor em todas as camadas.
- **Falha fechada.** Sem usuário, papel ou tenant, ou com papel desconhecido, tudo é negado.
- **Sem segredo antes da hora.** Verificações que não envolvem senha vêm antes das que envolvem.
- **Mínimo de dado em cada lugar.** O navegador, o log, a auditoria, o CSV e a IA recebem só o que precisam, com a mesma máscara.
- **Tudo auditável.** Toda decisão do sistema, inclusive as negadas, vira uma linha consultável.

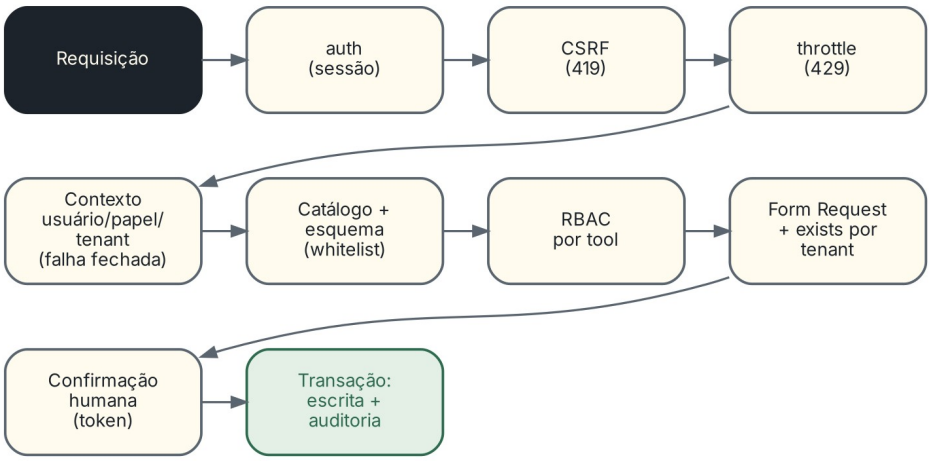


Figura 13 — Camadas que uma requisição de escrita atravessa até a gravação.

8.2 Ameaças e proteções

Tabela 14 — Ameaças, proteções e testes

Ameaça	Proteção	Onde	Teste que comprova
Injeção de prompt / tool inventada pelo modelo	Whitelist estática; argumentos conferidos contra o esquema; chave fora do esquema = resposta inválida; tool fora do catálogo negada e auditada só com nomes de argumentos	`interpretar()`, `executarTool()`	`test_tool_inventada_pelo_modelo_nao_executa_nada`; `test_argumentos_invalidos_nao_executam_tool`; `test_tool_inventada_com_cpf_e_salario_grava_so_os_nomes_dos_argumentos`
Acesso a dados de outra empresa (IDOR / tenant)	Tenant sempre do servidor; services exigem `int $tenantId`; regras `exists` reescritas para o tenant; registro de outro tenant responde como inexistente	`ContextoUsuario`, services, `escoparPorTenant()`	`test_tenant_enviado_pelo_modelo_e_ignorado`; `test_registro_de_outro_tenant_ou_na_lixeira_responde_igual_a_inexistente`; `test_service_com_o_tenant_de_outra_empresa_nao_encontra_nem_exclui`
Escalada de papel	RBAC por tool na proposta, no Revisar e na confirmação; falha fechada	`papelAtualPermite()`	`test_rbac_e_rechecado_na_confirmacao`; `test_rbac_e_reaplicado_no_revisar_e_de_novo_na_confirmacao`; `test_rbac_papel_desconhecido_nega_tudo`
Força bruta de senha	Login: 5/min por e-mail + IP. Card: 5 erros por admin digitado e 10 por usuário, contados antes de conferir	`AppServiceProvider`, `ReautenticacaoAdmin`	`test_login_bloqueia_a_sexta_tentativa_no_mesmo_minuto`; `test_a_tentativa_conta_antes_de_conferir_a_senha`
Usar o card para descobrir senhas	Papel e tenant conferidos antes da senha; mesma resposta com senha certa ou errada para quem não tem permissão	`AgenteController::confirmar()`	`test_usuario_rebaixado_recebe_o_mesmo_negado_com_senha_certa_ou_errada_sem_gastar_tentativa`
XSS	Blade só com `{{ }}`; JS só `createElement`/`textContent`; a IA nunca gera HTML	views, `partials/bancada.blade.php`	`test_erros_e_email_sao_escapados_e_senha_nao_e_reexibida` (JS: revisão de código)
CSRF	Middleware padrão do grupo `web`; cabeçalho `X-CSRF-TOKEN` em todo `fetch`; 419 em português	`layouts/app.blade.php`	`test_layout_envia_credenciais_ao_login_real_com_csrf`; bloqueio conferido no teste manual
Injeção de fórmula no CSV	Apóstrofo antes de `=`, `+`, `-`, `@`, também depois de separador ou aspas	`PlanilhaCsv::celula()`	`test_celula_que_comeca_com_formula_ganha_apostrofo`
Vazamento de dado pessoal	Máscara única; logs só com metadados; senha só na requisição; histórico só com frases fixas	`AuditoriaService`, `ConversaAgente`	`AuditoriaMascaraTest` (15 testes); `test_a_senha_nao_aparece_na_resposta_log_auditoria_sessao_historico_cache_nem_na_ia`

Ameaça	Proteção	Onde	Teste que comprova
Confirmação duplicada (corrida)	Token de uso único com `Cache::add`; um desfecho por proposta	`ConfirmacoesPendentes::concluir()`	`test_m1_confirmar_e_cancelar_simultaneos_so_o_primeiro_desfecho_vale`; `test_rnf02_token_so_vale_uma_vez`
Valor adulterado no navegador depois do Revisar	A confirmação usa os valores guardados na sessão; o corpo só leva o token	`AgenteController::confirmar()`	`test_valor_alterado_no_navegador_depois_do_revisar_e_ignorado`
Exclusão acidental	Revisar + segurar; bloqueio por dependência; lixeira; arquivamento; exclusão física só manual com senha	`ExclusaoSegura`, tools da lixeira	`GovernancaTest::test_rf09_*`, `test_rf11_rf12_*`; `test_excluir_e_so_para_admin`
Abuso de custo e de uso	Limites de taxa por rota (seção 8.4)	rotas `web`	`test_rnf10_a_11a_chamada_no_mesmo_minuto_recebe_429`; `test_limite_de_cinco_exportacoes_por_minuto`

8.3 Dados pessoais (LGPD)

Uma só máscara, definida em `AuditoriaService`, é aplicada em todos os lugares por onde um dado pessoal poderia sair: auditoria, resposta de consulta, formulário do card, CSV e histórico da conversa.

Tabela 15 — Máscara de dados sensíveis

Campo	Regra	Exemplo de saída
`cpf`	Mostra só os 5 últimos dígitos	`***.***.789-01`
`documento` com 11 dígitos	É CPF: mesma máscara do `cpf`	`***.***.789-01`
`documento` com 14 caracteres	É CNPJ, dado público de empresa: passa intacto	`12.345.678/0001-90`
`salario`	Nada do valor aparece	`[redigido]`
Valor que não é texto nem número	Redigido	`[redigido]`
Senha de admin	Só existe na requisição de confirmação; nunca em sessão, log, auditoria, cache ou IA	—

O administrador vê o salário em reais **apenas no card** de cadastro ou atualização de funcionário, porque precisa conferir o valor antes de gravar; em todo o resto (tabela, histórico, auditoria e CSV) o valor continua redigido. O CSV de funcionários sai sem CPF e sem salário.

8.4 Autenticação, sessão e limites

- Login com o `Auth` nativo do Laravel: `session()->regenerate()` no login e `invalidate()` no logout. Não há cadastro público; os usuários locais vêm do seeder

ou do comando `sentinel:preparar-acesso`, que mostra a senha aleatória só na criação.

- O logout dá desfecho às propostas abertas e limpa a memória da conversa.
- Na máquina da apresentação: `APP_DEBUG=false` (não expor stack traces) e `SESSION_ENCRYPT=true` (sessão cifrada).

Tabela 16 — Limites de taxa

Limitador	Limite	Chave
`login`	5 por minuto	e-mail + IP
`agente` (chama a IA)	10 por minuto	usuário
`agente-sem-ia` (revisar, cancelar, editar, formulário, consultar)	30 por minuto	usuário
`auditoria`	60 por minuto	usuário
`exportar`	5 por minuto	usuário
Senha de admin no card	5 erros por admin digitado; 10 por usuário, por janela	usuário + HMAC do e-mail

8.5 Ambiente e dependências

Tabela 17 — Ambiente e dependências

Item	Situação	Ação
PHP	A máquina de desenvolvimento usa PHP 8.4.25. As falhas críticas de outubro de 2026 (CVE-2026-109618 e CVE-2026-109619) exigem 8.4.27 ou mais	Atualização pedida ao TI do SENAI (o antivírus do laboratório bloqueia a instalação). Fazer também na máquina da apresentação
`session.upload_progress.enabled`	Ligado por padrão (CVE-2026-115560)	Desligado (`=0`) no `php.ini` da máquina principal; repetir nas demais
`composer audit`	Limpo desde `ae79781`	`symfony/http-foundation` (CVE-2026-108082) e `league/commonmark` atualizados
Pacotes npm	Ondas recentes de pacotes comprometidos no registro npm	Instalar sempre com `npm ci` (respeita o `package-lock.json`). O Sentinel não roda servidor Node em produção

> **Fora do escopo do protótipo (registrado, não descartado)**
>
> A lista completa do que fica para quando o projeto sair de protótipo (2FA, recuperação de senha, verificação de e-mail, tabela de tenants com chaves estrangeiras, observabilidade, hardening de servidor) está no Notion, na página "Checklist final — antes de sair de protótipo acadêmico".

9. O código em detalhe

Esta seção mostra trechos reais do repositório (revisão `ae79781`) que materializam as garantias descritas acima. Os trechos estão resumidos onde indicado por `// ...`; o restante é cópia fiel.

9.1 Uma tool no catálogo

Código 1 PHP · app/Services/AiAgentService.php, linhas 1057–1073

```
$this->definirTool(
    nome: 'consultar_clientes',
    descricao: 'Consulta clientes cadastrados, opcionalmente filtrando por parte do nome. Ação
de leitura — não exige confirmação.',
    tipo: 'leitura',
    risco: 'leitura',
    acao: 'consultar',
    entidade: 'Cliente',
    papelMinimo: 'leitura',
    propriedades: [
        'nome' => ['type' => 'string', 'description' => 'Parte do nome do cliente.'],
    ],
    obrigatorios: [],
    executar: fn (array $a) => $this->clienteService->buscarPorFiltro($a['nome'] ?? null,
$a['tenant_id']),
    mensagemErro: 'Não consegui consultar os clientes agora. Tente novamente.',
    colunas: ['id', 'nome', 'documento', 'email', 'telefone'],
    editavelPor: 'atualizar_cliente',
),
```

O que faz. Declara, num só lugar, tudo o que define uma ferramenta: o que a IA enxerga (nome, descrição e esquema JSON em `propriedades`), quem pode usar (`papelMinimo`), a classe de risco (que vira a cor da borda do card), a ação registrada na auditoria, as colunas devolvidas à tela, qual tool edita cada linha (`editavelPor`) e o executor.

Por que assim. Uma fonte única impede divergência entre o que a IA vê, o que o RBAC permite e o que a tela mostra. Repare que o esquema **não tem** `tenant_id`: o executor recebe o tenant do servidor, então o modelo não tem como pedir dados de outra empresa.

9.2 A chamada ao provedor e a medição do tempo

Código 2 PHP · app/Services/AiAgentService.php, linhas 148–185 (resumido)

```
// B1 (RNF04): tempo da chamada à IA, do envio até a resposta (ou a falha) do provedor.
$inicio = hrtime(true);
$decorrido = fn () => (int) round((hrtime(true) - $inicio) / 1e6);

try {
    $resposta = Http::withHeaders(['api-key' => $apiKey])
        ->timeout(30)
        ->post("{$endpoint}/openai/v1/chat/completions", [
            'model' => $deployment,
            'messages' => [
                ['role' => 'system', 'content' => 'Você é o assistente do Sentinel, ...'],
                ...$historico,
                ['role' => 'user', 'content' => $mensagem],
            ],
            'tools' => $this->tools(),
            'tool_choice' => 'auto',
        ]);
} catch (\Throwable $e) {
    // ... classifica timeout ou falha de conexão
    return $this->falha($timeout ? 'provider_timeout' : 'provider_connection_error', /* ...
*/);
}

$tempoIaMs = $decorrido();

// Tudo o que esta resposta auditar (leitura, negado, tool fora do catálogo) leva o tempo da
IA.
return $this->detalhesAuditoria->com(['tempo_ia_ms' => $tempoIaMs],
    fn () => $this->interpretar($resposta, $tempoIaMs));
```

O que faz. Faz a única chamada HTTP do sistema ao Microsoft Foundry: prompt de sistema, histórico curto, mensagem do usuário e a lista de tools derivada do catálogo. Cronometra a chamada com `hrtime` (relógio monotônico, imune a ajustes de hora) e coloca o tempo no escopo da auditoria.

Por que assim. Isolar a integração em uma classe atende à regra de arquitetura (trocar de provedor é mexer em um ponto) e torna o RNF04 mensurável sem espalhar código de medição pelo sistema. A chamada é stateless: nada fica guardado no provedor entre um pedido e outro.

9.3 Conferindo o que o modelo respondeu

Código 3 PHP · app/Services/AiAgentService.php, linhas 273–293 (resumido)

```
// Valida todas as chamadas antes de processar a primeira; nunca converte JSON inválido em
consulta ampla.
foreach ($toolCalls as $pedido) {
    // ...
    try {
        $objeto = json_decode($f['arguments'], false, 32, JSON_THROW_ON_ERROR);
    } catch (\JsonException) {
        return $this->respostaForaDoFormato($resposta->status());
    }
    $args = (array) $objeto;
    // Compatibilidade defensiva: tenant é sempre removido e derivado do backend.
    unset($args['tenant_id']);
    $schema = $this->catalogo()[$f['name']]['definicao']['function']['parameters']
['properties'] ?? null;
    if ($schema !== null && array_diff(array_keys($args), array_keys($schema)) !== []) {
        return $this->respostaForaDoFormato($resposta->status());
    }
}
```

O que faz. Antes de executar qualquer coisa, confere todas as chamadas de ferramenta que o modelo devolveu: o JSON precisa ser válido e ser um objeto, e nenhuma chave pode estar fora do esquema da tool. O `tenant_id` é descartado mesmo que o modelo o envie.

Por que assim. Um JSON quebrado nunca pode virar "consulta sem filtro" (que devolveria tudo). Chave inesperada é sinal de alucinação ou de injeção de prompt; o sistema prefere responder "não entendi" a adivinhar.

9.4 Revisar: validar sem gravar

Código 4 PHP · app/Http/Controllers/AgenteController.php, linhas 151–188 (resumido)

```
$tool = $rascunho['tool'];
$revisados = $this->aiAgentService->aplicarValores($tool, $rascunho['argumentos'], $valores);

$recusa = $this->aiAgentService->revisar($tool, $revisados,
ConfirmacoesPendentes::propostos($rascunho),
    ConfirmacoesPendentes::origem($rascunho), ConfirmacoesPendentes::tempoIa($rascunho));
// ...
if ($recusa !== null) {
    // O rascunho guarda o que já foi editado: na próxima revisão, sensível em branco mantém o
editado.
    $this->pendentes->atualizarRascunho($request->session(), $proposta, $rascunho,
$revisados);
    return response()->json([...$recusa, 'proposta' => $proposta, 'formulario' =>
$formulario], 422);
}

return response()->json([
    'tipo' => 'revisada',
    'proposta' => $proposta,
    'token' => $this->pendentes->emitirToken($request->session(), $proposta, $rascunho,
$revisados),
    'risco' => $this->aiAgentService->risco($tool),
]);
```

O que faz. Aplica os valores que o humano editou sobre a proposta guardada na sessão (só os campos do formulário) e valida com RBAC, tenant e as regras do Form Request, **sem gravar**. Se houver erro, devolve o formulário com a frase de cada campo; se não houver, emite o token de segurar.

Por que assim. Separar "revisar" de "confirmar" garante que o humano veja o formulário validado antes de o botão de segurar existir, o que combate a fadiga de confirmação. E a confirmação usa exatamente os valores revisados, guardados no servidor.

9.5 Confirmar: sem segredo antes da hora

Código 5 PHP · app/Http/Controllers/AgenteController.php, linhas 207–255 (resumido)

```
$pendente = $this->pendentes->retirarRevisada($request->session(), $token); // uso único
// ...
if (! $this->pendentes->concluir($proposta)) {
    return response()->json(['tipo' => 'erro', 'mensagem' => 'Esta confirmação já foi
usada.'], 409);
}
$decorrido = $this->pendentes->decorrido($pendente);

// Fora do prazo: antes da espera mínima, ou vencido (12 s do token ou os 5 min absolutos do
rascunho).
if ($decorrido < ConfirmacoesPendentes::ESPERA_MINIMA_S || $this->pendentes-
>expirou($pendente)) {
    // ... audita "expirado" ou "recusado" e responde 422
}

// Ordem: primeiro o que não envolve segredo (proposta revisada e no prazo, acima; papel e
tenant,
// aqui). Sem permissão, a senha nem é conferida nem gasta tentativa.
if ($this->aiAgentService->exigeReautenticacao($pendente['tool'])
    && $this->aiAgentService->podeExecutar($pendente['tool'])) {
    $autorizacao = $this->reautenticar($request);
    // ...
}
```

O que faz. Retira a proposta revisada da sessão (uso único), marca o desfecho de forma atômica, confere a janela de 2 a 12 s e só então, se a tool exigir e o usuário tiver permissão, confere a senha de administrador.

Por que assim. A ordem importa. Se a senha fosse conferida primeiro, um usuário sem permissão poderia usar o card para testar senhas de administradores. Com a ordem atual, ele recebe a mesma negação com senha certa ou errada e não gasta tentativas.

9.6 Uso único do token

Código 6 PHP · app/Services/ConfirmacoesPendentes.php, linhas 226–229

```
public function concluir(string $proposta): bool
{
    return Cache::add("agente:token_usado:{$proposta}", true,
        now()->addMinutes(max(1, (int) config('session.lifetime'))));
}
```

O que faz. Grava uma marca no cache somente se ela ainda não existir. `Cache::add` é atômico no armazenamento, então, entre duas requisições simultâneas, só uma recebe `true`.

Por que assim. Duas requisições podem ler a mesma sessão ao mesmo tempo (por exemplo, confirmar e cancelar em sequência rápida). Sem a marca atômica, a mesma proposta poderia ter dois desfechos. A marca é por proposta, não por token: dois tokens da mesma proposta nunca geram duas gravações.

9.7 Execução e auditoria na mesma transação

Código 7 PHP · app/Services/AiAgentService.php, linhas 470–493

```
$argumentos['tenant_id'] = $this->contexto->tenantId();

try {
    $resultado = $this->detalhesAuditoria->com([], fn () => DB::transaction(function () use
($tool, $definicao, $argumentos) {
        $retorno = $definicao['executar']($argumentos);

        $this->auditar(
            $tool,
            $definicao['acao'],
            $definicao['entidade'],
            $retorno instanceof Model ? (int) $retorno->getKey() : null,
            $argumentos,
            'sucesso',
            null,
            true,
        );

        return $retorno;
    }));
} catch (ToolRecusadaException $e) {
    // Regra de negócio: a mensagem já é uma frase amigável escrita para o usuário.
    $this->auditar($tool, /* ... */ 'recusado', $e->getMessage(), true);
```

O que faz. Sobrescreve o tenant com o do usuário logado e executa a tool e o registro de auditoria dentro de `DB::transaction`. Recusas de regra de negócio viram "recusado" com uma frase amigável.

Por que assim. Se gravar a auditoria falhar, a escrita é desfeita: não existe gravação sem rastro. Isso é comprovado por `test_escrita_e_auditoria_sao_atomicas_se_o_audit_falha_a_escrita_e_desfeita`.

9.8 RBAC com falha fechada

Código 8 PHP · app/Services/AiAgentService.php, linhas 1848–1855

```
private function papelAtualPermite(string $papelMinimo): bool
{
    $hierarquia = (array) config('sentinel.papeis', []);   // ['leitura', 'operador', 'admin']
    $atual = array_search($this->contexto->papel(), $hierarquia, true);
    $minimo = array_search($papelMinimo, $hierarquia, true);

    return $atual !== false && $minimo !== false && $atual >= $minimo;
}
```

O que faz. Compara a posição do papel do usuário com a do papel mínimo da tool na hierarquia configurada.

Por que assim. Se o papel do usuário ou o papel mínimo não estiverem na lista (erro de digitação, papel removido), `array_search` devolve `false` e a função nega. O erro de configuração resulta em "não pode", nunca em "pode tudo". A checagem roda na proposta, no Revisar e na confirmação.

9.9 Regras exists presas ao tenant

Código 9 PHP · app/Services/AiAgentService.php, linhas 1589–1605

```
private function escoparPorTenant(array $regras, int $tenantId): array
{
    return array_map(function ($regra) use ($tenantId) {
        // ...
        return array_map(function (string $parte) use ($tenantId) {
            if (! str_starts_with($parte, 'exists:')) {
                return $parte;
            }
            [$tabela, $coluna] = array_pad(explode(',', substr($parte, 7)), 2, 'id');

            return Rule::exists($tabela, $coluna)->where('tenant_id', $tenantId)-
>whereNull('deleted_at');
        }, explode('|', $regra));
    }, $regras);
}
```

O que faz. Reescreve, em tempo de execução, toda regra `exists:tabela,coluna` dos Form Requests para exigir o tenant atual e um registro fora da lixeira.

Por que assim. Sem isso, uma nota fiscal poderia apontar para o cliente de outra empresa só informando o id dele. A reescrita centralizada protege todos os Form Requests de uma vez.

9.10 Máscara LGPD

Código 10 PHP · app/Services/AuditoriaService.php, linhas 18 e 98–128 (resumido)

```
private const MASCARAS = ['cpf' => 'mascararCpf', 'salario' => 'redigir', 'documento' =>
'mascararDocumento'];

public static function mascararCamposSensiveis(array $parametros): array
{
    return collect($parametros)->map(function ($valor, $campo) {
        $metodo = self::MASCARAS[$campo] ?? null;
        return $metodo === null ? $valor : self::$metodo($valor);
    })->all();
}

private static function mascararDocumento(mixed $valor): mixed
{
    if ($valor === null) {
        return null;
    }
    if (! is_scalar($valor)) {
        return self::REDIGIDO;
    }
    return strlen(preg_replace('/\D/', '', (string) $valor)) === 11 ?
self::mascararCpf($valor) : $valor;
}
```

O que faz. Mapeia cada campo sensível para a sua regra de máscara. O campo `documento` decide pelo número de dígitos se é CPF (mascarado) ou CNPJ (público).

Por que assim. Ter uma fonte única evita que um lugar novo (por exemplo, o CSV) esqueça de mascarar. Tipos inesperados (listas, objetos) são redigidos em vez de passarem crus.

9.11 Higiene do histórico da conversa

Código 11 PHP · app/Services/ConversaAgente.php, linhas 65–81 (resumido)

```
private function sanitizar(string $texto): string
{
    // Credenciais não são necessárias aos fluxos de cadastro disponíveis.
    if (preg_match('/senha|password|api.?key|app.?key|csrf|cookie|session.?id|bearer/i',
$texto)) {
        return '[Mensagem com credenciais omitida do histórico]';
    }
    // ... troca a chave da API e a APP_KEY, se aparecerem, por '[credencial omitida]'
    $texto = preg_replace('/\b\d{2}\.?\d{3}\.?\d{3}\/?\d{4}-?\d{2}\b/', '[documento omitido]',
$texto);
    $texto = preg_replace('/\b\d{3}\.?\d{3}\.?\d{3}-?\d{2}\b/', '[documento omitido]',
$texto);
    $texto = preg_replace('/\b(sal[aá]rios?|remunera[cç](?:[ãa]o|[oõ]es))\b(\D{0,30}?)(?:R\$\
s*)?\d(?:[\d.,]*\d)?/iu',
        '$1$2[valor omitido]', $texto);
    return mb_substr($texto, 0, 1000);
}
```

O que faz. Antes de uma mensagem entrar no histórico que volta para a IA, remove credenciais, CNPJ, CPF e o valor que acompanha "salário" ou "remuneração", e corta em 1.000 caracteres.

Por que assim. O histórico é reenviado ao provedor a cada pedido; quanto menos dado pessoal ele carrega, menor a exposição. As limitações conhecidas (por exemplo, "ganha 4500" sem a palavra salário) estão registradas na seção 14.

9.12 Limite atômico de tentativas de senha

Código 12 PHP · app/Services/ReautenticacaoAdmin.php, linhas 72–98 (resumido)

```
// O e-mail digitado entra na chave só como HMAC: se alguém digitar a senha no campo
// do e-mail, ela não fica legível no cache.
$porUsuario = "reautenticacao:usuario:{$usuarioId}";
$porAdmin = "reautenticacao:admin:{$usuarioId}:".hash_hmac('sha256', Str::lower(trim($email)),
(string) config('app.key'));

// A tentativa conta ANTES de conferir a senha (incremento atômico no cache): pedidos em
// paralelo não passam todos pela conferência antes de o primeiro erro ser contado.
$tentativasDoUsuario = RateLimiter::hit($porUsuario, self::JANELA_S);
$tentativasNoAdmin = RateLimiter::hit($porAdmin, self::JANELA_S);

if ($tentativasDoUsuario > self::LIMITE_POR_USUARIO || $tentativasNoAdmin >
self::LIMITE_POR_ADMIN) {
    $this->devolverTentativa($porUsuario, $porAdmin);
    return ['falha' => 'limite'];
}

$admin = /* ... */ $this->adminQueConfere(trim($email), $senha);
if ($admin === null) {
    return ['falha' => 'invalido'];
}

// Acerto não conta: a tentativa volta.
$this->devolverTentativa($porUsuario, $porAdmin);
return ['admin' => (int) $admin->id];
```

O que faz. Conta a tentativa antes de conferir a senha, com dois limites (por usuário e por e-mail de admin digitado). Só um admin **do mesmo tenant** é aceito. O acerto devolve a tentativa.

Por que assim. Se a contagem viesse depois da conferência, vários pedidos em paralelo passariam todos antes de o primeiro erro ser registrado. O HMAC evita que uma senha digitada por engano no campo de e-mail fique legível no cache.

9.13 Bloqueio por dependência

Código 13 PHP · app/Services/Concerns/ExclusaoSegura.php, linhas 42–59 e 181–192

```
protected function excluirComBloqueio(Model $registro, array $relacoesQueBloqueiam): array
{
    $motivo = $this->motivoDeBloqueio($registro, $relacoesQueBloqueiam);
    if ($motivo !== null) {
        return ['bloqueado' => true, 'motivo' => $motivo];
    }
    // Uma marca de arquivado que sobrou faria o registro pular o prazo da lixeira
(RF09/RF11).
    if ($registro->arquivado_em !== null) {
        $registro->forceFill(['arquivado_em' => null])->save();
    }
    $registro->delete();   // soft delete
    return ['bloqueado' => false, 'registro' => $registro];
}

private function motivoDeBloqueio(Model $registro, array $relacoes): ?string
{
    foreach ($relacoes as $relacao) {
        if ($registro->{$relacao}()->exists()) {
            $rotulo = self::ROTULOS_RELACAO[$relacao] ?? "registros em {$relacao}";
            return "Não é possível excluir: existem {$rotulo} a este registro.";
        }
    }
    return null;
}
```

O que faz. Verifica as relações que impedem a exclusão e devolve o motivo em linguagem simples; sem dependência, faz o soft delete (lixeira).

Por que assim. O motivo é escrito para o usuário, não para o desenvolvedor, como pede o RF08. A limpeza da marca de arquivado evita um caso raro em que o registro pularia o prazo da lixeira.

9.14 CSV sem injeção de fórmula

Código 14 PHP · app/Services/PlanilhaCsv.php, linhas 53–64

```
$texto = (string) $valor;

if (is_int($valor) || is_float($valor) || preg_match('/^-?\d+(\.\d+)?$/D', $texto) === 1) {
    return $texto;
}

// UTF-8 inválido vira "?" antes das regex (com /u, elas devolveriam null e a célula sumiria).
$texto = mb_scrub(strtr($texto, ["\r" => ' ', "\n" => ' ', "\t" => ' ']), 'UTF-8');
$texto = (string) preg_replace('/([,;][\s"\x{00A0}]*)([=+\-@])/u', "\$1'\$2", $texto);

return preg_match('/^[\s"\x{00A0}]*[=+\-@]/u', $texto) === 1 ? "'".$texto : $texto;
```

O que faz. Neutraliza células que começariam com `=`, `+`, `-` ou `@` (inclusive depois de separador, espaço ou aspas) colocando um apóstrofo na frente. Números puros passam intactos.

Por que assim. Um nome de cliente como `=HYPERLINK(...)` viraria uma fórmula executável ao abrir o CSV no Excel. O CSV também sai com BOM e separador `;`, para abrir corretamente no Excel em português.

9.15 Segurar para confirmar, no navegador

Código 15 JavaScript · resources/views/partials/bancada.blade.php, linhas 183–184, 867–881 e 937–952 (resumido)

```
const DURACAO_HOLD_MS = 2000;
const DURACAO_IDLE_MS = 10000;

function passoHold() {
    if (!segurando) return;
    const decorrido = performance.now() - inicioHold;
    const pct = Math.min(100, (decorrido / DURACAO_HOLD_MS) * 100);
    fill.style.width = pct + '%';
    grande.textContent = Math.floor(pct) + '%';
    if (pct >= 100) {
        segurando = false;
        confirmarAcao();
        return;
    }
    rafId = requestAnimationFrame(passoHold);
}

btnConfirmar.addEventListener('mousedown', iniciarHold);
btnConfirmar.addEventListener('touchstart', iniciarHold);
btnConfirmar.addEventListener('keydown', (e) => {
    if ((e.key === 'Enter' || e.key === ' ') && !e.repeat) { e.preventDefault();
iniciarHold(); }
});
// mouseup, mouseleave, blur, touchend, touchcancel e keyup chamam soltarHold()
```

O que faz. Anima o preenchimento a cada quadro (`requestAnimationFrame`) enquanto o botão é mantido; ao chegar a 100%, envia só o token ao servidor. Mouse, toque e teclado iniciam e encerram o gesto.

Por que assim. O gesto contínuo de 2 s evita confirmações por clique acidental, e a acessibilidade por teclado é obrigatória. A segurança, porém, **não depende** do navegador: o servidor recusa confirmações antes de 2 s ou depois de 12 s, mesmo que alguém altere este JavaScript.

10. Modelo de dados e auditoria

O banco tem seis tabelas de domínio, a tabela de usuários e a tabela de auditoria. Todas as tabelas de domínio têm `tenant_id` (a empresa), `deleted_at` (lixeira) e `arquivado_em` (arquivamento).

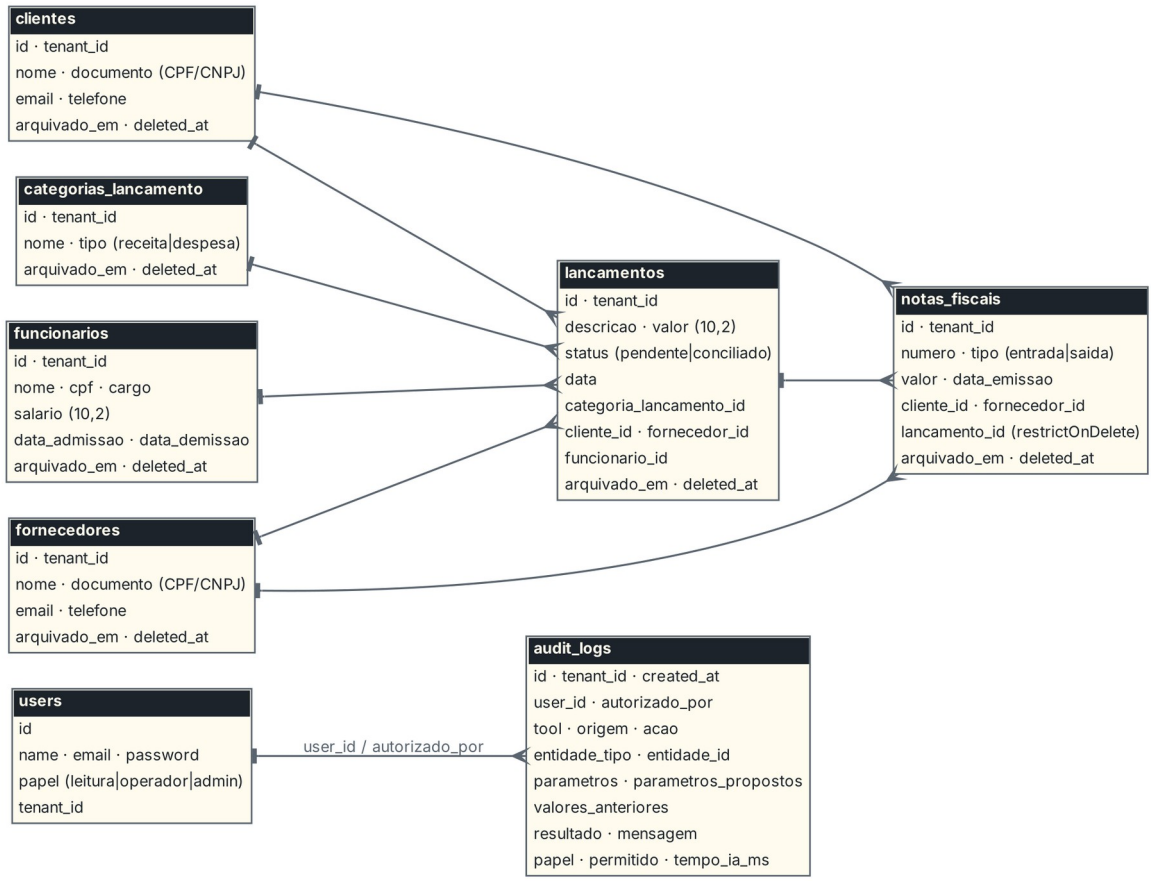


Figura 14 — Modelo de dados: entidades de domínio, usuários e auditoria.

Tabela 18 — Tabelas do domínio

Tabela	Colunas relevantes	Relações
`lancamentos`	`descricao`, `valor` (10,2), `status` (pendente/conciliado), `data`	Categoria, cliente, fornecedor e funcionário (opcionais)
`categorias_lancamento`	`nome`, `tipo` (receita/despesa)	Tem lançamentos
`clientes` e `fornecedores`	`nome`, `documento` (CPF ou CNPJ), `email`, `telefone`	Têm lançamentos e notas fiscais
`funcionarios`	`nome`, `cpf`, `cargo`, `salario` (10,2), `data_admissao`, `data_demissao`	Têm lançamentos
`notas_fiscais`	`numero`, `tipo` (entrada/saída), `valor`, `data_emissao`	Cliente, fornecedor e lançamento, com `restrictOnDelete`

Tabela	Colunas relevantes	Relações
`users`	`name`, `email`, `password` (hash), `papel`, `tenant_id`	Autor e autorizador na auditoria

10.1 A tabela de auditoria

`audit_logs` é a memória do sistema. Ela responde quem fez o quê, por qual caminho, com qual permissão, o que a IA propôs, o que o humano confirmou, o que havia antes e quanto tempo a IA levou.

Tabela 19 — Colunas de audit_logs

Coluna	Conteúdo
`user_id`	Quem estava logado
`autorizado_por`	O administrador que digitou a senha (restaurar e excluir de vez)
`tool`	Nome da tool (ou `[tool redigida]` quando o modelo inventou uma)
`origem`	`agente`, `formulario` ou `manual`
`acao`	`consultar`, `criar`, `atualizar`, `excluir`, `restaurar`, `excluir_definitivo`, `exportar` ou `desconhecida`
`entidade_tipo`, `entidade_id`	O registro afetado
`parametros`	O que foi executado (mascarado)
`parametros_propostos`	O que a IA propôs (mascarado; vazio quando veio sem IA)
`valores_anteriores`	O "antes" dos campos alterados (mascarado)
`resultado`, `mensagem`	O desfecho (seção 7.2) e a frase mostrada ao usuário
`papel`, `permitido`	O papel no momento e se a ação foi permitida
`tempo_ia_ms`	Tempo da chamada à IA (RNF04)
`tenant_id`, `created_at`	Empresa e momento

> **Proposto × confirmado**
>
> A coluna `parametros_propostos` permite mostrar, na demonstração, um caso em que a IA sugeriu um valor e o humano o corrigiu antes de confirmar. É a evidência concreta do "Jarvis com freio".

10.2 Migrations

Tabela 20 — Migrations em ordem de execução

#	Migration	O que faz
1–3	`create_users_table`, `create_cache_table`, `create_jobs_table`	Estrutura padrão do Laravel (usuários, cache, sessões)
4	`2026_09_08_173654_create_lancamentos_table`	Lançamentos (Fase 0)

#	Migration	O que faz
5–9	`2026_09_09_2222xx_create_*`	Categorias, clientes, fornecedores, funcionários e notas fiscais
10	`add_relacionamentos_e_governanca_to_lancamentos_table`	Chaves estrangeiras, soft delete e arquivamento em lançamentos
11	`create_audit_logs_table`	Tabela de auditoria
12	`add_papel_e_permitido_to_audit_logs_table`	Papel e permissão na auditoria (RNF01)
13	`add_papel_e_tenant_id_to_users_table`	Papel e tenant no usuário (RF10)
14	`add_proposto_autorizacao_tempo_e_origem_to_audit_logs_table`	Proposto, quem autorizou, tempo da IA e origem
15	`add_valores_anteriores_to_audit_logs_table`	Valor anterior dos campos alterados

11. Interface e acessibilidade

A interface tem duas zonas com regras diferentes. A **vitrine** (antes do login) tem fundo escuro e pode ter movimento: um farol 3D que gira com a rolagem. A **bancada** (login e tudo depois dele) tem fundo creme e **nenhuma animação decorativa**, só o preenchimento do segurar e barras de progresso. O protótipo de baixa fidelidade continua no Figma; as telas finais seguem a página "Direção visual" do Notion e a skill `sentinel-visual` do repositório.

Tabela 21 — Tokens de cor (todas as combinações passam WCAG AA)

Token	Cor	Uso
`noite`	#0A0E13	Fundo da vitrine
`gelo`	#A8C8E6	Destaque sobre fundo escuro
`creme`	#FFFCEF	Fundo da bancada
`tinta` / `tinta-2`	#1D232A / #5B6570	Texto principal e secundário; borda de risco neutro (leitura)
`aco`	#2E5B85	Único destaque sobre creme: links, foco, rótulos
`aviso`	#8A5A12 (faixa #F6EBD0)	Borda de escrita pendente; avisos
`erro`	#A3412E (faixa #F5E0D9)	Borda de exclusão; mensagens de erro
`sucesso`	#2F6B4F (faixa #E4EFE6)	Confirmação concluída

11.1 Sinalização de risco

Todo card de ação do agente tem uma borda de 2 px na cor do risco: neutra para leitura, âmbar para escrita e vermelha para exclusão. A cor nunca está sozinha: título, texto da etapa e ícone repetem a mesma informação. Tipografia: Sora nos títulos, IBM Plex Sans no corpo e IBM Plex Mono em números, códigos e CNPJ, com fallback para Segoe UI.

11.2 Acessibilidade

- Estado nunca comunicado só por cor ou só por animação (RNF07).
- Mensagens do agente em região `aria-live`; o leitor de tela ouve o início e o resultado do gesto de segurar, não a porcentagem.
- Segurar funciona com teclado (Enter ou Espaço mantidos); nada depende de passar o mouse por cima.
- Foco visível em tudo que é clicável; área de toque mínima de 44 × 44 px; link "pular para o conteúdo".
- Contorno dos campos com contraste de 5,8:1 (o mínimo exigido é 3:1).
- Funciona em 360 px de largura sem rolagem horizontal; tabelas largas rolam dentro do próprio contêiner.
- `prefers-reduced-motion` respeitado na vitrine.

12. Qualidade e testes

A estratégia de qualidade do projeto é provar cada garantia de governança com um teste automatizado (RNF09). A suíte roda com `php artisan test`, em SQLite na memória e com o provedor de IA simulado (`Http::fake`), então é rápida, gratuita e determinística: as respostas da "IA" são escritas pelo próprio teste, inclusive as maliciosas e as malformadas.

Tabela 22 — Testes automatizados por arquivo (343 métodos; 398 casos com os conjuntos de dados)

Arquivo de teste	Métodos	O que cobre
`FormularioAgenteTest`	36	Formulário no card (RF04), Revisar, valores do navegador ignorados, rascunho de 5 min
`AgenteTest`	26	Leitura, escrita confirmada (RNF02), limite de taxa (RNF10), falha do provedor
`LixeiraTest`	25	Excluir, restaurar, excluir de vez, dependências, papel e senha
`ConfirmacaoDesfechoTest`	23	Cancelar, expirar, varredura, logout, corrida entre confirmar e cancelar
`AtualizarTest`	19	Atualizar por id ou nome, botão Editar, alteração concorrente, outro tenant
`AutenticacaoTest`	17	Login, logout, limite de tentativas, contexto do usuário
`AuditoriaConsultaTest`	16	Consulta da auditoria: só admin, por tenant, filtros, detalhe mascarado
`AuditoriaMascaraTest`	15	Máscara de CPF, salário e documento em todos os formatos
`AgenteGovernancaTest`	15	Tool inventada, tenant ignorado, RBAC repetido, atomicidade da auditoria
`ExportarCsvTest`	14	CSV: anti-fórmula, BOM, máscara, limite, tenant, auditoria
`AtualizarStatusLancamentoTest`	14	Conciliar e voltar a pendente
`FuncionarioNotaFiscalToolsTest`	14	Tools de funcionário (só admin) e nota fiscal
`ValorAnteriorTest`	13	Valor anterior mascarado; tool inventada grava só nomes
`ReautenticacaoCardTest`	12	Senha de admin: ordem, limites, nunca vaza, nunca 500
`AgenteCorrecoesTest`	12	Argumentos inválidos, operação não suportada
`GovernancaTest`	10	RF08, RF09, RF11, RF12, RNF03 nos services
Outros 13 arquivos	62	Tools de cliente, fornecedor e categoria; tempo da IA; modo manual; telas; casca; login; vitrine; acesso local

12.1 Técnicas usadas

- **Respostas adversárias da IA:** o teste simula o modelo pedindo uma tool inexistente, mandando `tenant_id` de outra empresa, JSON quebrado, chaves extras, resposta truncada ou filtrada.

- **Sabotagem das proteções:** durante a construção, cada proteção foi desligada de propósito para confirmar que o teste correspondente falhava.
- **Vazamento procurado em todos os lugares:** a senha de admin é procurada na resposta, no log, na auditoria, na sessão, no histórico, no cache e no corpo enviado à IA.
- **Concorrência:** confirmar e cancelar a mesma proposta em sequência produz um único desfecho.

12.2 Teste manual

Durante a construção (Fases 4 e 5) a rede de segurança foi a suíte automática. O teste manual completo, por papel (leitura, operador e admin), com a Azure real e numa máquina limpa do laboratório, está planejado para a Fase 6, a partir do roteiro do relatório final (`RELATORIO_FINAL_FASES_4_5.md`). Itens que só o teste manual comprova: a parte visual do RNF07, o bloqueio de CSRF (o Laravel o desliga nos testes), o comportamento em 360 px e os tempos reais da IA (RNF04).

13. Requisitos e rastreabilidade

Os requisitos são os da versão 1.0. Para cada um, a tabela indica a situação e a evidência no código e nos testes (arquivos em `tests/Feature/`).

13.1 Requisitos funcionais

Tabela 23 — Rastreabilidade dos requisitos funcionais

ID	Requisito	Situação	Evidência
RF01	Interpretar comandos em linguagem natural e mapeá-los para operações válidas	Feito	`processar()`/`interpretar()`, catálogo de 37 tools · `AgenteTest`, `AgenteGovernancaTest`, `AgenteCorrecoesTest`
RF02	Consultar com filtros em linguagem natural; tabela editável e exportável	Feito	Tools `consultar_*`, `editavelPor`, `ConsultasExportaveis`, `PlanilhaCsv` · `AtualizarTest`, `ExportarCsvTest`
RF03	Executar ações em massa somente com confirmação explícita	Especificado (13.3)	Hoje toda escrita é unitária e confirmada; vários pedidos numa mensagem: só o primeiro é atendido, com aviso · `test_varios_pedidos_numa_mensagem_so_o_primeiro_e_feito_e_a_resposta_avisa`
RF04	Cadastro e atualização por formulário pré-preenchido, invocado pela IA	Feito	`FormularioTool`, `AgenteController::revisar()` · `FormularioAgenteTest` (36), `AtualizarTest`
RF05	Telas de CRUD tradicionais como via alternativa	Feito	`TelasController`, `/agente/consultar`, `/agente/formulario` · `ModoManualTest`, `TelasTest`
RF06	Registrar em log toda ação executada, cancelada ou negada	Feito	`AuditoriaService`, `executarTool()`, `auditarDesfecho()` · `AgenteGovernancaTest`, `ConfirmacaoDesfechoTest`
RF07	Consulta e filtragem do log por usuários autorizados	Feito	`ConsultaAuditoria`, `AuditoriaController`, tela `/auditoria` · `AuditoriaConsultaTest` (16)
RF08	Bloquear exclusão com dependências ativas, com motivo simples	Feito	`ExclusaoSegura` · `GovernancaTest::test_rf08_*`, `LixeiraTest`
RF09	Exclusão vai para lixeira com retenção configurável	Feito	SoftDeletes + `retencao_lixeira_dias` (90) · `GovernancaTest::test_rf09_*`
RF10	Restaurar da lixeira com senha de administrador, registrando na auditoria	Feito	`restaurar_*`, `ReautenticacaoAdmin`, `autorizado_por` · `LixeiraTest`, `ReautenticacaoCardTest` (12)
RF11	Após o prazo, mover para arquivado (invisível, mantido no banco)	Feito (depende do agendador do SO)	`sentinel:arquivar-expirados`, agendado diariamente · `GovernancaTest::test_rf11_rf12_*`
RF12	Exclusão física só manual, por administrador	Feito	`excluir_definitivo_*` (admin + senha) · `GovernancaTest`, `LixeiraTest`

13.2 Requisitos não funcionais

Tabela 24 — Rastreabilidade dos requisitos não funcionais

ID	Requisito	Situação	Evidência
RNF01	RBAC aplicado às tools da IA, não só às telas	Feito	`papel_minimo` por tool, `papelAtualPermite()` · `test_rbac_hierarquia_leitura_operador_admin`, `test_rbac_papel_desconhecido_nega_tudo`
RNF02	Toda escrita exige confirmação humana explícita	Feito	Proposta → Revisar → token → segurar 2 s → confirmar · `AgenteTest::test_rnf02_*` (4)
RNF03	Validação exclusivamente no backend	Feito	Form Requests via `validar()` + `escoparPorTenant()` · `test_valor_alterado_no_navegador_depois_do_revisar_e_ignorado`
RNF04	Tempo de resposta mensurável e dentro de um limite (referência: 10 s)	Em aberto	Mensurável: `tempo_ia_ms` em cada desfecho · `TempoIaTest` (8). O limite não é imposto (timeout de 30 s); decisão adiada para a próxima fase, com os números do teste manual
RNF05	Com a IA fora do ar, o CRUD continua operável	Feito	Modo manual · `test_com_a_azure_fora_do_ar_lista_busca_cadastra_e_exporta`
RNF06	Isolamento de tenant em todas as camadas, inclusive nas tools	Feito	`ContextoUsuario`, tenant sobrescrito, `exists` escopado · `test_tenant_enviado_pelo_modelo_e_ignorado`, `ExportarCsvTest`
RNF07	Estado de ação pendente em pelo menos dois canais	Feito (visual a conferir)	Borda + título + texto + `aria-live` + popup · `test_confirmacao_pendente_traz_o_risco_escrita_da_tool`; parte visual no teste manual
RNF08	Expirar cancela só a ação pendente	Feito	`VarrerConfirmacoesVencidas` · `test_varredura_nao_mexe_na_proposta_ainda_valida`
RNF09	Governança coberta por testes automatizados	Feito	398 casos; arquivos de governança listados na seção 12
RNF10	Limite de taxa na rota da IA	Feito	`throttle:agente` (10/min) e demais limitadores · `test_rnf10_a_11a_chamada_no_mesmo_minuto_recebe_429`

13.3 RF03: ações em massa por plano de ações

Na construção, toda escrita ficou unitária: um registro por vez, sempre com Revisar e segurar. Para atender ao RF03 sem abrir mão das garantias, a equipe especificou o padrão **planejar → o humano aprova → executar**: o agente apresenta um plano listando cada ação e o impacto, e só depois da aprovação explícita o sistema executa.

Situação

O RF03 está especificado. A implementação será planejada junto com a nova data de congelamento de funcionalidades, que está em aberto.

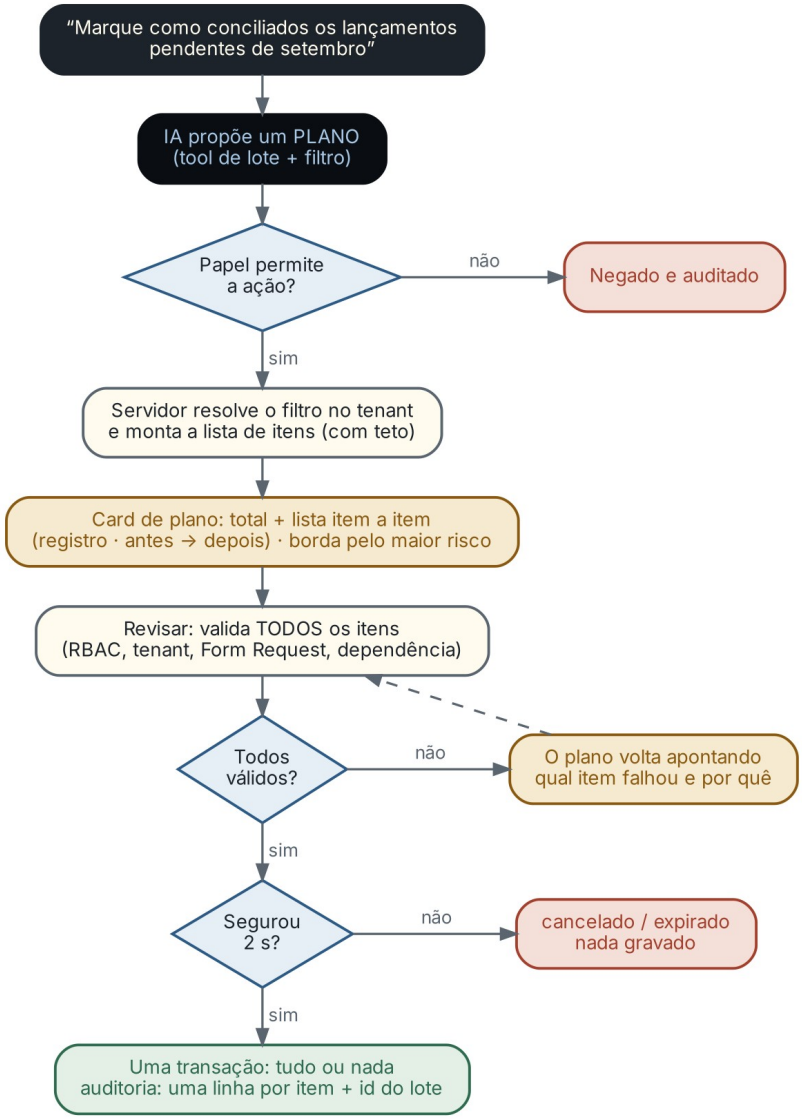


Figura 15 — RF03 especificado: plano de ações com revisão de todos os itens e confirmação única.

- **Card de plano:** total de itens ("12 lançamentos vão para conciliado"), lista item a item com o registro e o antes → depois, e borda pela cor do maior risco do lote.
- **Revisar:** valida todos os itens (RBAC, tenant, Form Request, dependência); se um falhar, o plano volta apontando qual item e por quê.
- **Segurar uma vez:** a gravação é feita numa única transação, tudo ou nada.
- **Auditoria:** uma linha por item, todas com o mesmo identificador de lote.
- **Teto de itens por plano** e resumo do impacto sempre visível, contra a fadiga de confirmação.

14. Limitações reconhecidas

As limitações abaixo são conhecidas e foram aceitas conscientemente. Nenhuma delas permite gravar dado sem confirmação ou acessar dado de outro tenant.

Tabela 25 — Limitações e riscos aceitos

#	Limitação	Impacto e motivo da aceitação
1	Campo sensível opcional (ex.: CPF no documento) não pode ser apagado pelo formulário: em branco mantém o valor anterior	Evita apagar dado sem querer; para limpar, é preciso outro caminho
2	Campo sensível com erro de validação volta em branco no formulário	O valor digitado não é devolvido ao navegador (LGPD)
3	No histórico da conversa, o salário só é escondido logo depois de "salário" ou "remuneração"	"ganha 4500" e o segundo número de "entre 3000 e 4500" passam; CPF e CNPJ já são omitidos
4	Duas abas gravando a sessão ao mesmo tempo podem perder uma proposta ou um id de exportação	Efeito: "proposta indisponível"; nada é gravado indevidamente
5	A página 404 de usuário logado não mostra o menu	Só navegação; o link para o agente aparece
6	A lista de escolha (vários registros com o mesmo nome) não é auditada	Não grava nada; a ação escolhida depois é auditada
7	Recusado e erro de atualização sem `entidade_id` na coluna	O id aparece nos parâmetros
8	Senha de admin errada encerra a proposta	É preciso pedir de novo; reduz tentativas por proposta
9	`lockForUpdate` não tem efeito no SQLite dos testes	No MySQL a trava vale; o uso único do token é testado
10	CNPJ sem máscara e filtro "nome" sem máscara na auditoria de consulta	CNPJ é público; um CPF digitado no filtro apareceria
11	CSV: número negativo puro sem apóstrofo; telefone "+55" com apóstrofo	Decisão: `-150.00` é número; o apóstrofo no telefone é inofensivo
12	Modo manual: tool fora do catálogo recebe 422 sem linha na auditoria	Não executa nada; o limite de 30/min continua valendo
13	RF03 (ações em massa) não implementado	Especificado na seção 13.3
14	RNF04: o limite de 10 s é medido, não imposto	Decisão em aberto para a próxima fase
15	RF11 depende do Agendador de Tarefas do Windows chamar `schedule:run`	Sem ele, o arquivamento só roda manualmente
16	`tenant_id` sem chave estrangeira	Ainda não existe a tabela de empresas
17	Restaurar não desfaz efeitos externos já ocorridos (ex.: uma notificação enviada)	Limitação inerente; o Sentinel não envia notificações hoje
18	Dependência de serviço de terceiros e custo por uso da API	Mitigadas pelo modo manual e pelos limites de taxa

15. Próximos passos

15.1 Antes da entrega

- Definir a nova data de congelamento de funcionalidades e decidir se o RF03 (plano de ações) entra antes dela.
- Teste manual completo por papel, com a Azure real e numa máquina limpa do laboratório; anotar os tempos da IA para decidir o RNF04.
- Atualizar o PHP para 8.4.27 ou mais nas máquinas do laboratório e na da apresentação; `session.upload_progress.enabled=0` em todas.
- Na máquina da apresentação: `APP_DEBUG=false`, `SESSION_ENCRYPT=true` e o Agendador do Windows chamando `schedule:run` (RF11).
- Remover as telas de amostra antes da entrega; dados de demonstração próprios; plano B em vídeo.

15.2 Depois do TCC

- Tabela de empresas (tenants) com chaves estrangeiras e migração para Supabase PostgreSQL (checklist no `CLAUDE.md`).
- Validação real de documentos (CPF, CNPJ alfanumérico, PIS) e verificação de e-mail.
- Camada offline (PWA ou aplicativo com banco local) para a falta total de conectividade.
- Entrada por voz, dada a familiaridade do público com mensagens de áudio.
- Camada preditiva sobre os dados estruturados e auditados que o próprio uso gera (seção 3.4).
- Itens de produção listados no Notion: 2FA, recuperação de senha, observabilidade e hardening.

Referências

MICROSOFT. Anomaly Detector: What's New / FAQ. Microsoft Learn, Azure AI Services, 2025–2026.

MICROSOFT. Azure OpenAI in Microsoft Foundry: function calling (Chat Completions). Microsoft Learn, 2026.

NIELSEN, J. AI as a Paradigm Shift in User Interfaces. Nielsen Norman Group, 2023.

OWASP. CSV Injection. OWASP Foundation.

OWASP. Top 10 for Large Language Model Applications. OWASP Foundation, 2025.

SEBRAE. Mapa de Empresas: 2º quadrimestre de 2025. Ministério do Empreendedorismo, da Microempresa e da Empresa de Pequeno Porte, 2025.

SEBRAE. Pesquisa TIC 2025: Transformação Digital nos Pequenos Negócios. Sebrae/PR, 2025.

SEBRAE. Pesquisa de Maturidade Digital dos Pequenos Negócios no Brasil. Sebrae/PR, 2024–2025.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD).

W3C. Web Content Accessibility Guidelines (WCAG) 2.2. 2023.

YAO, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022.

Apêndice A. Glossário

Tabela 26 — Termos usados neste documento

Termo	Significado no Sentinel
Agente	O modelo de linguagem somado à camada de orquestração que confere e executa o que ele decide
Tool (ferramenta)	Operação registrada no catálogo que o modelo pode pedir; ele escolhe, o servidor executa
Catálogo / whitelist	A lista fechada de tools; o que não está nela não existe para o modelo
Tenant	A empresa dona dos dados; cada usuário pertence a um tenant e só enxerga os dados dele
RBAC	Controle de acesso por papel (leitura, operador, admin), aplicado a cada tool
Proposta / rascunho	Escrita pedida pela IA ou por um formulário, guardada na sessão e ainda não gravada
Revisar	Etapa que valida o formulário sem gravar e libera o botão de segurar
Token de segurar	Identificador de uso único emitido pelo Revisar e exigido na confirmação
Segurar para confirmar (hold-to-confirm)	Manter o botão Confirmar pressionado por 2 s para autorizar a escrita
Desfecho	O resultado final de uma proposta: sucesso, negado, recusado, cancelado, expirado ou erro
Soft delete / lixeira	Exclusão que só marca o registro (`deleted_at`), mantendo-o restaurável
Arquivado	Registro que passou do prazo da lixeira: invisível nas telas, mantido no banco
Falha fechada	Na dúvida (dado ausente ou inválido), negar em vez de permitir
Modo manual	Uso das telas de Cadastros e Lixeira sem chamar a IA
Human-in-the-loop	Modelo de supervisão em que a decisão de escrita é sempre de um humano